Modern .NET Development (2026)

A Guide for Experienced C# Developers

Steve Snively

2026

Contents

CHAPTER 1

The Modern .NET Landscape

Chapter Purpose

This chapter rebuilds the map.

If your working model of .NET was formed around Visual Studio 2019, .NET Framework or early modern .NET, Windows Server, IIS, and SQL Server, the core skills still matter. C# still matters. HTTP still matters. SQL still matters. Production discipline still matters.

What changed is the shape of the system around those skills.

Modern .NET development is no longer centered on one Windows server running one IIS-hosted application connected to one SQL Server. That model still exists, and many businesses still run it successfully, but it is no longer the default mental model for new professional .NET systems. Today, the application is often one part of a larger platform made of source control, automated validation, containers, cloud services, managed identity, deployment pipelines, observability, and sometimes AI services.

The purpose of this chapter is not to teach Docker, Kubernetes, CI/CD, cloud hosting, or AI in detail. Those subjects come later. The purpose is to show where they live on the map, why they became important, and how they relate to the enterprise .NET knowledge you already have.

By the end of this chapter, you should be able to explain how modern .NET systems fit together at a high level and why the ecosystem changed after the Visual Studio 2019 and .NET 5 era.

Where This Fits

This is the first chapter because every later chapter depends on the same orientation: modern .NET is not just a runtime upgrade. It is a different way of thinking about the application lifecycle.

A traditional enterprise .NET system might have looked like this:

```
Developer workstation
        |
        v
Visual Studio build
        |
        v
Manual or scripted deployment
        |
        v
Windows Server + IIS
        |
        v
SQL Server
```

A modern .NET system often looks more like this:

```
Developer workstation or cloud dev environment
        |
        v
```

```
Git repository and pull request
        |
        v
Automated build and test pipeline
        |
        v
Container image or deployable package
        |
        v
Cloud platform, container host, or orchestrator
        |
        v
Managed databases, caches, queues, identity, and AI services
        |
        v
Logs, metrics, traces, alerts, and deployment feedback
```

The application code is still central, but it is no longer the whole story. Modern professional development treats the path from source code to production as part of the system.

This book follows that path. First, it rebuilds the mental model. Then it builds applications. Then it introduces Linux, containers, deployment, cloud platforms, production operations, AI, architecture, and interview preparation.

Connection to the Reader's Existing Model

You already know the old landmarks.

You know that an IIS application pool isolates a web application process. You know that configuration files can change behavior without recompiling. You know that SQL Server is both a database engine and an operational responsibility. You know that Windows services run background work. You know that a load-balanced pair of Windows servers introduces session, deployment, and diagnostic concerns that a single server does not.

Modern .NET keeps many of those ideas but changes where the boundaries are.

An ASP.NET Core process still receives HTTP requests and produces responses, but IIS may be only one possible front end, or not involved at all. A container can package the application and its runtime dependencies in a way that feels a little like a disciplined deployment folder, but it also behaves like a standardized process boundary. A cloud platform can replace some server administration work with managed services, but it introduces new ownership questions around identity, cost, networking, and reliability.

The analogy to traditional Windows hosting is useful because the same concerns remain:

- Where does the process run?
- How is it configured?
- How does it connect to the database?
- How is it deployed?
- How do you know whether it is healthy?
- What happens when it fails?

The analogy breaks down because modern systems distribute those concerns across more tools. Instead of one administrator configuring one server, a team may define build rules in source control, publish a container image, provision infrastructure with code, deploy to a managed platform, and monitor behavior through telemetry.

That sounds like more moving parts because it is. The tradeoff is that those parts make repeatability, scale, automation, and team collaboration easier when the system grows.

Layer 1 — Conceptual Model

The modern .NET landscape is the collection of runtimes, tools, platforms, and operating practices used to build and run .NET applications today.
At the center is .NET itself: the runtime, SDK, base class libraries, languages, ASP.NET Core, and related frameworks. Around it is the professional delivery system: source control, automated tests, containerization, cloud hosting, configuration, secrets, observability, security, and operations.
The important conceptual shift is this:

```
Old center of gravity: the server
New center of gravity: the delivery platform
```

In the older model, the server was often the durable center of the application. You installed software on it, configured IIS, patched the operating system, managed local folders, copied releases into place, and inspected logs on the machine.
In the modern model, the server may be temporary, abstract, replicated, or hidden behind a managed service. The durable center becomes the combination of source code, build pipeline, deployment definition, container image, configuration system, managed data services, and telemetry.
This does not mean servers disappeared. It means developers are expected to understand platforms that create, replace, scale, observe, and secure runtime environments.
Modern .NET exists to let .NET applications participate naturally in that world:

- cross-platform runtime support instead of Windows-only assumptions;
- command-line and SDK-based workflows instead of IDE-only workflows;
- ASP.NET Core hosting that can run behind IIS, Nginx, cloud load balancers, or container platforms;
- container images for repeatable deployment;
- cloud-native configuration, logging, health checks, and dependency injection;
- libraries and abstractions that integrate with modern identity, telemetry, messaging, and AI services.

It does not solve every problem by itself. Modern .NET does not automatically make an application scalable, secure, observable, or maintainable. It gives you the building blocks and platform alignment to do those things deliberately.

Layer 2 — System Relationships

A modern .NET application participates in a larger system. Understanding the relationships is more important at first than memorizing product names.
The source repository is the system of record for code and increasingly for configuration templates, build definitions, deployment manifests, tests, and architecture notes. In older environments, some of that knowledge lived in server settings or deployment runbooks. In modern environments, teams try to move as much of it as practical into versioned artifacts.
The build system converts source code into something runnable. That might be a published folder, a NuGet package, a container image, or a deployable artifact. The build is not just compilation. It usually includes restore, static checks, tests, packaging, and sometimes vulnerability checks.
The runtime environment is where the application process executes. It might be a developer workstation, a Windows service, IIS, a Linux VM, a container host, a platform-as-a-service environment, or Kubernetes. ASP.NET Core is flexible enough to run in several of these environments, but each environment changes the operational responsibilities.
The application dependencies provide the services the code needs to do useful work: SQL Server, caches, queues, file storage, identity providers, external APIs, observability systems, and AI models. In modern systems, more of these dependencies are managed services rather than software installed on the same server as the application.

The deployment system moves validated changes into an environment. A manual copy to an IIS folder is a deployment system, just a fragile one. A modern pipeline makes the steps explicit, repeatable, reviewable, and observable.
The feedback system tells the team what happened after deployment. Logs, metrics, traces, health checks, alerts, dashboards, and error reports are not extras. They are the way a team sees a system that may be spread across multiple processes, services, regions, and managed platforms.
The main failure boundary also changed. In a simple IIS application, the failure boundary was often the web server, application pool, database, or network. Those still matter. Modern systems add more boundaries: container startup, image registry access, cloud identity, service discovery, queue backlogs, rate limits, region outages, deployment pipeline failures, and telemetry gaps.
The professional .NET developer does not need to be the expert owner of every boundary, but they do need to recognize them.

Layer 3 — Core Mechanics

This chapter keeps mechanics small. Later chapters go deep.
The first mechanic is the modern .NET release model. Microsoft now ships major .NET releases annually, and releases alternate between Long Term Support (LTS) and Standard Term Support (STS). As of July 22, 2026, Microsoft lists .NET 10 as the current LTS release, with .NET 9 as an STS release and .NET 8 as an LTS release still in support. This matters because choosing a target framework is also choosing a support and upgrade rhythm.
The second mechanic is side-by-side installation. Modern .NET versions can be installed alongside one another. That is different from the old feeling of a machine-wide .NET Framework version being part of the Windows server's personality. A developer machine, build agent, or server may have multiple SDKs or runtimes installed.
The third mechanic is SDK-centered development. The .NET SDK supplies the command-line tools used to create, build, test, publish, and package projects. Visual Studio remains important, especially for many Windows-based enterprise teams, but the SDK makes the workflow portable across local machines, CI agents, containers, and cloud build environments.
The fourth mechanic is ASP.NET Core's hosting flexibility. An ASP.NET Core application is a process that can run behind different front ends. IIS can still be part of the story, but it is no longer the only natural host. The same application model can run on Windows, Linux, in a container, or on a managed cloud platform.
The fifth mechanic is containerization. A container image packages the application with the runtime environment expected by the application. Microsoft publishes official .NET container images for different scenarios, including SDK images for building and ASP.NET/runtime images for running applications.
The sixth mechanic is platform integration. Modern .NET applications commonly use dependency injection, configuration providers, structured logging, health checks, and telemetry hooks. These are not decorative framework features. They exist because modern applications must be configured, deployed, monitored, and changed repeatedly across environments.
The seventh mechanic is AI integration. AI is now part of the modern .NET landscape, but in this book it is not magic dust sprinkled over every chapter. For now, treat AI as two related developments: AI-assisted development, where tools help developers write and reason about code, and AI-enabled applications, where .NET systems call models, embedding services, or agent frameworks as part of product behavior.

Layer 4 — Developer Workflow

A typical modern .NET workflow is still recognizable.
You create a project. You write C#. You run it locally. You debug it. You test it. You check it into source control. You deploy it. You fix bugs.
The difference is that the workflow is less centered on one workstation and one server.

At a high level, the workflow looks like this:

```
Create or clone repository
        |
        v
Restore dependencies
        |
        v
Run and debug locally
        |
        v
Add tests and commit changes
        |
        v
Open pull request
        |
        v
Automated build and test checks
        |
        v
Package or publish
        |
        v
Deploy to an environment
        |
        v
Observe behavior and iterate
```

In Visual Studio 2019-era development, it was common for the IDE to hide many of these steps. Modern teams still use IDEs, but they also expect the same steps to run outside the IDE. That is why command-line workflows, project files, source-controlled pipeline definitions, and repeatable local setup have become more important.
For this first chapter, the useful commands are discovery commands rather than implementation commands:

```
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

On Windows PowerShell, the same commands apply:

```
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

These commands answer basic orientation questions: which SDKs are installed, which runtimes are available, and what environment the .NET CLI sees.
The details of creating projects, building APIs, testing, containerizing, and deploying come later. For now, the important workflow shift is that modern .NET development is designed to be repeatable across local development, automated builds, and production environments.

Layer 5 — Production Usage

Production is where the modern landscape becomes real.
In a traditional IIS deployment, production concerns were often concentrated on the server:

- IIS configuration;

- application pool identity;
- installed .NET runtime;
- web.config settings;
- Windows Event Log;
- file permissions;
- SQL Server connection strings;
- load balancer behavior.

Modern .NET production systems still have equivalent concerns, but they are distributed across the application, platform, and delivery process.

Configuration is usually environment-specific. Local development might use user secrets, environment variables, or local configuration files. Production might use a cloud configuration service, secret manager, injected environment variables, or platform-level settings. The principle is the same as protecting connection strings in older systems, but the mechanisms are broader.

Security extends beyond application login. Authentication and authorization still matter, but so do service identities, secret rotation, container image scanning, dependency updates, cloud permissions, TLS, and deployment access.

Reliability is no longer just "keep the server up." It includes health checks, restart behavior, retries, timeouts, queue handling, horizontal scaling, database failover, and safe deployment strategies.

Deployment becomes a repeatable process. A production release should not depend on a person remembering which files to copy or which server setting to change. The goal is not automation for its own sake. The goal is reducing variation between releases.

Observability replaces server guessing. When an application runs in multiple instances, containers, or managed platforms, logging into the server is often not the first diagnostic move. You need structured logs, metrics, traces, health checks, alerts, and dashboards to understand what the system is doing.

Scaling moves from buying a bigger server to choosing the right boundary. Sometimes the answer is a larger database. Sometimes it is more application instances. Sometimes it is caching, background processing, or queue-based work. Sometimes the correct answer is to simplify the design.

Persistence remains central. SQL Server is still a major part of many .NET systems, including modern ones. The difference is that it may be hosted as a managed database, run in a container for local development, accessed through Entity Framework Core or Dapper, and deployed alongside migration automation.

Cost becomes a design concern. In a traditional environment, cost was often hidden behind infrastructure budgets. In cloud environments, architectural decisions can affect monthly bills directly. A chatty service, oversized database, unnecessary Kubernetes cluster, or inefficient AI call can have a visible cost.

The recurring production distinction is simple:

```
Local development optimizes for fast feedback.
Production optimizes for reliability, security, repeatability, and visibility.
```

Modern .NET gives you tools that can support both, but the design must be intentional.

Layer 6 — Tradeoffs and Alternatives

Modern does not always mean better. It means better aligned with current professional expectations and platform realities.

Use modern .NET practices when the application needs cross-platform hosting, cloud deployment, automated delivery, containerized environments, strong observability, frequent change, or integration with managed services.

Do not assume every application needs every modern tool. A small internal application may not need Kubernetes. A batch utility may not need a distributed architecture. A stable line-of-business system may benefit more from automated tests and clear deployment scripts than from a wholesale platform migration.

Simpler alternatives remain valid:

- IIS hosting can still be appropriate for some ASP.NET Core applications.
- A single well-designed application can be better than a premature set of microservices.
- A managed platform-as-a-service option can be simpler than running Kubernetes.
- SQL Server can remain the right database.
- Manual approval gates can be appropriate even inside automated pipelines.

More advanced alternatives exist for systems that need them:

- container orchestration;
- infrastructure as code;
- service meshes;
- event-driven architecture;
- distributed tracing;
- multi-region deployment;
- AI agents and retrieval-augmented generation.

The common overengineering mistake is treating the modern stack as a checklist. Docker, Kubernetes, Redis, queues, cloud platforms, and AI services each solve specific problems. They also add operational responsibility. A strong modern .NET developer knows both sides of that tradeoff.
The better question is not "Are we using the newest thing?" It is "Which system boundary is causing pain, and which tool makes that boundary clearer, safer, or easier to change?"

Layer 7 — Interview Perspective

Interviewers do not usually expect a returning .NET developer to be a deep expert in every modern platform tool. They do expect an accurate map.
Concepts commonly tested:

- the difference between .NET Framework, .NET Core, and modern .NET;
- why ASP.NET Core is not tied to IIS in the same way classic ASP.NET was;
- what containers solve and what they do not solve;
- why CI/CD exists;
- how cloud hosting changes deployment and operations;
- the difference between logs, metrics, and traces;
- why microservices are not automatically better than monoliths;
- how AI services fit into ordinary application architecture.

Representative questions:

- "What changed in .NET after .NET Framework?"
- "Why would a .NET team run an application on Linux?"
- "What problem does Docker solve for a .NET API?"
- "When would you choose a managed cloud service instead of a VM?"
- "What is the difference between deploying code and operating a production system?"
- "How would you modernize an older IIS-hosted application without rewriting everything?"

A strong answer connects concepts. For example:

> "Docker does not make the application scalable by itself. It packages the application and its runtime environment in a repeatable way. Scaling depends on where the container runs, whether the app is stateless, how configuration and secrets are handled, and what the database can support."

Common misconceptions:

- "Modern .NET means cloud only."
- "ASP.NET Core requires Linux."
- "Containers replace deployment pipelines."
- "Kubernetes is the default place to run every production app."
- "Microservices are the modern version of layered architecture."
- "AI integration means replacing normal application design."

Small design scenario:
You inherit an ASP.NET application hosted on Windows Server with IIS and SQL Server. Releases are manual, configuration differs between environments, and production issues are diagnosed by remote desktop access and log files on the server.
A good modernization discussion would not begin with "move it to Kubernetes." It would begin by identifying the current pain:

- Is deployment unreliable?
- Is the application hard to test?
- Are environments inconsistent?
- Is the server difficult to patch?
- Are production failures hard to diagnose?
- Is scaling actually required?

Then it would propose a sequence: get the code building consistently, add automated tests, clarify configuration, improve logging and health checks, choose a deployment target, and only then consider containers, cloud services, or orchestration if they solve a real problem.

Hands-On Lab

Objective:
Create a personal map of the modern .NET landscape and inspect the .NET installation on your machine.
Prerequisites:

- A development machine with the .NET SDK installed.
- A terminal: Windows Terminal, PowerShell, Command Prompt, macOS Terminal, or a Linux shell.
- No application code is required yet.

Steps:

1. Open a terminal.
2. Run the following command:

   ```
   dotnet --info
   ```

3. Identify the installed SDK version or versions.
4. Identify the installed runtime version or versions.
5. Note the operating system and architecture reported by the command.
6. Run:

```
dotnet --list-sdks
```

7. Run:

```
dotnet --list-runtimes
```

8. Draw a simple architecture map for a modern .NET application using these boxes:

```
Developer
Source control
Build and test pipeline
Application package or container image
Runtime host
Database
Observability
Users
```

9. Add arrows showing how code moves from development to production and how production feedback returns to the team.

Expected results:

- You can name the .NET SDKs and runtimes installed on your machine.
- You can explain the difference between local development, build automation, deployment, runtime hosting, and production feedback.
- You have a first-pass map of the modern .NET system this book will fill in.

Validation commands:

```
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

Troubleshooting notes:

- If `dotnet` is not recognized, the SDK may not be installed or may not be on your `PATH`.
- If only runtimes are installed, you may be able to run .NET applications but not build them.
- If several SDKs are installed, that is normal. Modern .NET versions can live side by side.

Knowledge Check

1. Why is “modern .NET” more than a newer runtime version?
2. In what ways is a container similar to a disciplined deployment package, and where does that analogy break down?
3. Why did the center of gravity move from individual servers toward delivery platforms?
4. What production concerns remain the same whether an application runs under IIS, in a container, or on a cloud platform?
5. Why is Kubernetes not the natural starting point for every modernization effort?
6. How does AI-assisted development differ from building AI-enabled application features?
7. Why should a returning .NET developer understand CI/CD even before becoming a deployment specialist?
8. What does observability provide that server access alone does not?

Summary

Modern .NET development begins with the same durable skills you already have: C#, HTTP, SQL, configuration, deployment judgment, and production discipline. The landscape around those skills has changed.

The old model placed the Windows server and IIS near the center. The modern model places the application inside a delivery platform made of source control, automated validation, repeatable packaging, cloud or container hosting, managed dependencies, security boundaries, and production telemetry.

.NET adapted to that world by becoming cross-platform, SDK-centered, cloud-friendly, container-friendly, and better integrated with modern configuration, logging, dependency injection, hosting, and AI patterns.

The rest of this book fills in the map one layer at a time. First you will learn the major technologies and workflow. Then you will build .NET applications. Then you will run them on Linux and in containers. Then you will deploy, observe, secure, scale, and reason about them as production systems.

The goal is not to chase novelty. The goal is to become fluent again in the professional environment where modern .NET systems are built and operated.

Sources

- Microsoft Learn, ".NET releases, patches, and support": https://learn.microsoft.com/en-us/dotnet/core/releases-and-support
- Microsoft, ".NET Support Policy": https://dotnet.microsoft.com/en-us/platform/support/policy
- Microsoft Learn, "Microsoft .NET and .NET Core - Microsoft Lifecycle": https://learn.microsoft.com/en-us/lifecycle/products/microsoft-net-and-net-core
- Microsoft Learn, ".NET container images": https://learn.microsoft.com/en-us/dotnet/core/docker/container-images
- Microsoft Learn, "Develop .NET apps with AI features": https://learn.microsoft.com/en-us/dotnet/ai/overview

Further Reading

- Microsoft Learn, "What's new in .NET 10": https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview
- Microsoft Learn, "Install .NET on Windows": https://learn.microsoft.com/en-us/dotnet/core/install/windows
- Microsoft Learn, "AI apps for .NET developers": https://learn.microsoft.com/en-us/dotnet/ai/

CHAPTER 2

The Modern Technology Stack

Chapter Purpose

Chapter 1 rebuilt the map. This chapter labels the major landmarks.

The modern .NET stack is not a single product. It is a working combination of runtime, web framework, operating system, packaging model, deployment platform, data stores, source-control system, delivery pipeline, and increasingly AI services.

If you come from a Windows, IIS, SQL Server, and Visual Studio background, the older stack may have felt vertically integrated. Microsoft supplied the language, runtime, IDE, web server integration, database, operating system, and deployment target. You could build a serious enterprise application while staying almost entirely inside one vendor ecosystem.

That world still exists, but the center widened.

Modern .NET remains a Microsoft-led ecosystem, but professional .NET work now commonly touches Linux, Docker, Kubernetes, Redis, GitHub, cloud platforms, open-source libraries, managed services, and AI providers. Some of these tools were adopted because they solved problems Microsoft products already faced. Some became industry standards outside the Microsoft ecosystem and .NET had to meet developers where the industry had moved. That is not a weakness. It is one of the reasons modern .NET is viable in cloud-native systems.

The purpose of this chapter is to introduce the major technologies without teaching their implementation details yet. You should finish this chapter able to explain what each technology is for, how it relates to the technologies around it, and where it fits in a complete application system.

This chapter deliberately stays at the map level. Later chapters teach the mechanics.

Where This Fits

Chapter 1 described the shift from server-centered development to platform-centered development. Chapter 2 turns that platform into named components.

A typical modern .NET application stack might look like this:

```
Users
  |
  v
Web or API edge
  |
  v
ASP.NET Core application
  |
  +-- SQL Server or managed relational database
  |
  +-- Redis cache
  |
  +-- Queue, background worker, or external service
  |
  +-- AI service when the application needs model behavior
```

```
  |
  v
Logs, metrics, traces, and alerts

Source code -> GitHub -> CI/CD -> package or container image -> host platform
```

The technologies in the roadmap do not all sit at the same layer:

- .NET is the runtime and developer platform.
- ASP.NET Core is the web and API framework.
- Linux is a common operating-system host.
- Docker is a packaging and local runtime tool for containers.
- Kubernetes is an orchestration platform for running containers at scale.
- SQL Server is the relational database many enterprise .NET systems already know.
- Redis is a fast in-memory data store often used for caching and coordination.
- Cloud platforms provide managed compute, databases, identity, networking, and operations.
- GitHub is commonly the source-control and collaboration hub.
- CI/CD is the automated path from code change to validated artifact and, eventually, deployment.
- AI services provide model-backed behavior that applications can call like other external dependencies.

This chapter introduces all of them together so later chapters can zoom in without forcing you to learn the system vocabulary from scratch.

Connection to the Reader's Existing Model

The easiest way to understand the modern stack is to compare each layer to something familiar.

.NET is still the platform you write C# against. The difference is that modern .NET is cross-platform, SDK-centered, open-source, and released on a regular cadence. It is less tied to a particular Windows installation than .NET Framework felt.

ASP.NET Core still handles HTTP requests, routing, application services, configuration, logging, and responses. The familiar IIS mental model helps, but ASP.NET Core is not just "classic ASP.NET with new project files." It can run behind IIS, behind a Linux reverse proxy, inside a container, or on a managed cloud platform.

Linux is the operating system you are most likely to encounter when .NET code runs outside Windows. Think of it as another production host with different filesystem, process, permission, service, and shell conventions. You do not need to become a Linux administrator to be productive, but you do need enough fluency to read paths, inspect processes, understand permissions, and interpret logs.

Docker is closest to a disciplined deployment package plus a lightweight process boundary. It is not a virtual machine in the traditional Hyper-V sense. It packages an application and its runtime dependencies as an image, then runs that image as an isolated process called a container.

Kubernetes is closest to a large-scale operations control plane. If a load-balanced IIS farm answers "which server receives the request?", Kubernetes also asks "which containers should exist, where should they run, how should they be replaced, how are they reached, and how is desired state maintained?" That analogy is useful, but Kubernetes is broader and more abstract than a web farm.

SQL Server is already familiar. The modern shift is not that relational databases disappeared. It is that SQL Server may run on Windows, Linux, in a container for development, in Azure SQL Database, in Azure SQL Managed Instance, or on a VM. The database remains a strong consistency and persistence boundary.

Redis is not a replacement for SQL Server. Think of it as a very fast in-memory structure server that can hold cached data, counters, distributed locks, session-like state, streams, or temporary coordination data. It solves a different class of problems from a relational database.

Cloud platforms are the modern equivalent of renting a data center where many infrastructure concerns are exposed as APIs and managed services. They replace some hardware and server administration with service selection, identity, networking, cost management, and operational design.

GitHub is more than a remote Git folder. In many teams it is the collaboration surface for pull requests, code review, security scanning, issue tracking, automation, and package publishing.

CI/CD replaces memory-based release discipline with executable release discipline. If you previously relied on a build machine, deployment checklist, and human care, CI/CD moves more of that process into source-controlled automation.

AI services are external intelligence dependencies. From an architecture point of view, they are closer to calling a payment processor, search service, or document service than to adding an ordinary class library. They introduce latency, cost, security, prompt design, evaluation, and correctness concerns.

Layer 1 — Conceptual Model

The modern technology stack can be understood as five cooperating layers:

```
Application layer
  .NET, ASP.NET Core, C#, application libraries

Data and dependency layer
  SQL Server, Redis, queues, files, APIs, AI services

Runtime and packaging layer
  Windows, Linux, Docker, container images

Platform and operations layer
  Cloud platforms, Kubernetes, identity, networking, observability

Delivery layer
  GitHub, pull requests, CI/CD, artifact registries, release automation
```

The application layer is where most business code lives. This is where you write APIs, validation, business rules, background services, and integrations.

The data and dependency layer is where the application stores durable state, reads fast temporary state, communicates with other systems, and calls external capabilities.

The runtime and packaging layer answers the question, “What exactly runs, and what does it require from the machine?” This includes the OS, installed runtime or container image, process environment, ports, files, certificates, and startup behavior.

The platform and operations layer answers the question, “Where does it run, how is it secured, how does it scale, and how do we know it is healthy?”

The delivery layer answers the question, “How does a code change become a validated production change?”

No single layer is the whole stack. A developer who only knows the application layer may write good C# but struggle to diagnose production. A developer who only knows platform tools may ship elaborate infrastructure for a simple problem. Modern .NET fluency means understanding how the layers cooperate.

Layer 2 — System Relationships

Each technology has inputs, outputs, dependencies, ownership boundaries, and failure modes.

.NET takes source code, project files, NuGet packages, SDKs, and configuration as inputs. It produces assemblies, applications, packages, and runtime behavior. The development team usually owns the code and package choices; the platform or operations team may own installed runtimes, base images, and runtime policies.

ASP.NET Core takes HTTP requests, configuration, dependency registrations, and middleware as inputs. It produces HTTP responses, logs, metrics, and calls to dependencies. It fails when routing is wrong, configuration is missing, dependencies are unavailable, startup fails, or runtime exceptions escape.

Linux takes processes, files, users, groups, sockets, services, and environment variables as operating-system concepts. A .NET developer usually does not own the whole Linux estate, but they may own enough of the application process to read logs, understand paths, inspect environment variables, and troubleshoot container behavior.

Docker takes a Dockerfile, application output, base images, and configuration as inputs. It produces container images and containers. The development team often owns the application image. A platform team may own the registry, base image policy, runtime host, and security scanning.

Kubernetes takes declarative resource definitions that describe desired state. It attempts to keep the actual cluster state aligned with those definitions. The application team may own deployment definitions for its service; a platform team often owns the cluster, networking, ingress, policies, secrets integration, and shared observability.

SQL Server takes schema, queries, transactions, indexes, data files, memory, storage, and connections. It produces durable state and query results. Its failure boundaries include locking, blocking, disk, memory, schema migration, backup, recovery, permissions, and network access.

Redis takes keys, values, data structures, expiration rules, memory, and client connections. It produces fast reads, writes, and coordination behavior. Its failure boundaries include memory pressure, eviction policy, persistence configuration, clustering, and the risk of treating cached data as if it were durable relational truth.

Cloud platforms take resource definitions, subscriptions or accounts, permissions, regions, networks, and billing rules. They produce hosted services. Their failure boundaries include identity, quota, cost, region availability, misconfiguration, provider limits, and shared-responsibility misunderstandings.

GitHub takes commits, branches, pull requests, issues, workflow files, and repository permissions. It produces review history, source history, automation events, and collaboration records. Its failure boundaries include branch-policy gaps, secret leaks, broken workflows, weak review practices, and permissions that are too broad.

CI/CD takes repository events, workflow definitions, build agents, secrets, tests, package feeds, and deployment credentials. It produces build results, test reports, packages, images, deployments, and audit trails. Its failure boundaries include flaky tests, missing environment parity, credential problems, incomplete rollback plans, and automation that hides rather than reduces risk.

AI services take prompts, input data, context, model choices, safety settings, tools, embeddings, and credentials. They produce generated text, structured outputs, classifications, embeddings, or tool calls. Their failure boundaries include latency, cost, hallucination, data leakage, model drift, evaluation gaps, and ambiguous ownership of generated behavior.

The relationships matter because production failures rarely respect chapter boundaries. A slow request might involve ASP.NET Core routing, Redis cache misses, SQL Server query plans, cloud network latency, and missing telemetry. The stack is a system.

Layer 3 — Core Mechanics

The smallest useful example is a web API with a database, cache, and delivery pipeline:

```
GitHub repository
  |
  v
CI builds and tests the .NET solution
  |
```

```
  v
Docker image is produced
  |
  v
Cloud platform runs the ASP.NET Core container
  |
  +-- SQL Server stores orders
  |
  +-- Redis caches product lookups
  |
  +-- AI service summarizes support notes
  |
  v
Observability records logs, metrics, and traces
```

The core terminology starts here.

A runtime executes compiled application code. In modern .NET, the runtime is not the same thing as the SDK. The SDK builds and publishes applications; the runtime runs them.

A framework provides reusable application structure. ASP.NET Core is the web framework used to build HTTP APIs, web applications, real-time apps, and services.

An operating system hosts processes, files, networking, permissions, and resource limits. Windows and Linux can both host modern .NET applications.

A container image is a packaged filesystem and metadata template used to create containers. A container is a running instance of that image.

An orchestrator manages many containers across machines. Kubernetes is the dominant open-source orchestrator and is widely used in cloud-native environments, but it is not required for every application.

A relational database stores structured durable data using tables, relationships, indexes, transactions, and queries. SQL Server remains a major enterprise relational database and, according to DB-Engines in July 2026, ranked third overall among database management systems.

An in-memory data store keeps data primarily in memory for speed. Redis is a prominent example and ranked eighth overall in the July 2026 DB-Engines ranking.

A cloud platform provides infrastructure and application capabilities as services: compute, databases, storage, networking, identity, observability, and AI.

A repository stores source history. Git is the version-control system; GitHub is a collaboration platform built around Git repositories and automation.

Continuous integration means automatically building and testing code changes after they are committed or proposed. Continuous delivery and deployment extend that automation toward release artifacts and environments.

An AI service exposes model behavior through an API or SDK. In modern .NET, Microsoft.Extensions.AI provides common abstractions for interacting with models and related AI capabilities across providers.

Layer 4 — Developer Workflow

Chapter 3 will teach the modern workflow in more detail. For this chapter, the workflow is about recognizing which tool participates at each stage.

Early in development, you mostly touch .NET, ASP.NET Core, SQL Server or a local database, maybe Redis, and GitHub. You run locally, test quickly, and commit changes.

As the application grows, Docker often enters the workflow to make the local environment more repeatable. Instead of every developer installing the same database, cache, and supporting services by hand, a team can describe a local multi-container environment.

When the team starts sharing changes, CI becomes important. A pull request should not depend only on “it works on my machine.” It should trigger a build and tests in a clean environment.
When the team starts deploying repeatedly, CD becomes important. The question shifts from “Can we build it?” to “Can we safely move this exact validated artifact into an environment?”
As production needs grow, cloud services and observability become part of the daily workflow. Developers need to know how configuration reaches the application, where logs go, how database access is secured, how health is reported, and how deployments can be rolled back.
Kubernetes usually appears after these basics. It is most useful when a team needs a consistent platform for many containerized workloads, not when it is trying to learn containers for the first time.
AI services may appear in two places. AI coding assistants may help during development. AI application services may become runtime dependencies when the product needs summarization, classification, chat, extraction, embeddings, or agentic workflows.
Useful orientation commands:

```
dotnet --info
git --version
docker --version
kubectl version --client
```

On Windows PowerShell, the commands are the same:

```
dotnet --info
git --version
docker --version
kubectl version --client
```

These commands are not a setup requirement for this chapter. They simply show which parts of the stack are already present on your machine.

Layer 5 — Production Usage

In production, the technology stack becomes a set of responsibility boundaries.
.NET and ASP.NET Core define the application runtime boundary. Production questions include which .NET version is supported, whether the app is patched, how configuration is loaded, how logs are emitted, how health is exposed, and how dependency failures are handled.
Linux or Windows defines the host operating-system boundary. Production questions include patching, file permissions, service identity, certificates, network access, process limits, and diagnostic access.
Docker defines the packaging boundary. Production questions include base-image trust, image size, vulnerability scanning, registry access, immutable tags, startup behavior, and whether runtime configuration is separated from the image.
Kubernetes defines the orchestration boundary. Production questions include replicas, health probes, rolling updates, secrets integration, network policy, resource limits, storage, cluster ownership, and operational complexity.
SQL Server defines the durable data boundary. Production questions include backup and restore, high availability, transaction isolation, migration strategy, index health, query performance, permissions, and cost.
Redis defines the fast state boundary. Production questions include whether the data is disposable or durable, how memory is sized, how eviction works, whether clustering is needed, and how cache misses affect the database.
Cloud platforms define the managed-service boundary. Production questions include region choice, identity, networking, quotas, compliance, platform service-level agreements, cost controls, and shared responsibility.
GitHub and CI/CD define the delivery boundary. Production questions include who can approve changes, which checks are required, how secrets are protected, how artifacts are versioned, and how deployments are audited.

AI services define the model boundary. Production questions include data privacy, prompt injection, output validation, model selection, cost per call, evaluation, latency, fallback behavior, and observability of AI decisions.

The state-of-the-art direction across these areas is not "use more tools." It is platform engineering: make the correct path easy for teams. A mature organization gives developers paved paths for common tasks such as creating an API, adding a database, creating a pipeline, publishing a container, connecting to secrets, emitting telemetry, and deploying safely.

That is the modern production goal: not heroic deployment knowledge, but repeatable systems that make the safe thing normal.

Layer 6 — Tradeoffs and Alternatives

The modern stack is full of good tools, and good tools can still be wrong for the job.

.NET competes most often with ecosystems such as Java, Go, Node.js, Python, Rust, and Kotlin. .NET is strong for enterprise applications, APIs, cloud services, Windows integration, high-performance server workloads, and teams that value C# and mature tooling. It may not be the natural first choice when a team is already deeply invested in another ecosystem or when a specialized runtime dominates a domain.

ASP.NET Core competes with frameworks such as Spring Boot, Express, FastAPI, Django, Rails, Laravel, and Go HTTP frameworks. ASP.NET Core is strong for typed, high-performance APIs and enterprise web systems. It can be excessive for a tiny script-like endpoint, and it may be unfamiliar to teams centered on JavaScript or Python.

Linux competes with Windows Server as a host for .NET workloads. Linux is common for containers and cloud-native hosting. Windows remains appropriate for applications with Windows-specific dependencies, legacy COM integration, classic .NET Framework, or operational teams built around Windows tooling.

Docker competes with direct host installation, virtual machines, language- specific packaging, and platform-native deployment packages. Docker is useful when environment repeatability matters. It can be unnecessary for a simple internal utility or a platform-as-a-service deployment that already abstracts packaging well.

Kubernetes competes with simpler options: Azure App Service, Azure Container Apps, AWS Elastic Beanstalk, AWS ECS, Google Cloud Run, ordinary VMs, and managed platform services. Kubernetes is powerful when many services need a shared orchestration model. It is often overengineering for one or two modest applications.

SQL Server competes with PostgreSQL, MySQL, Oracle, SQLite, cloud-native managed databases, document databases, and analytical stores. SQL Server is a strong choice for enterprise relational workloads, T-SQL expertise, Microsoft tooling, and existing estates. PostgreSQL is a major open-source alternative; SQLite is excellent for embedded and local scenarios; document stores fit different data shapes.

Redis competes with in-process memory caches, Memcached, database caching, message brokers, and specialized streaming systems. Redis is useful when fast shared state or data structures matter. It is not a substitute for relational integrity, durable event logs, or careful database design.

Cloud platforms compete with on-premises hosting, colocation, private cloud, and hybrid designs. Azure is often natural for Microsoft-centered enterprises, AWS is broad and deeply established, and Google Cloud has strengths in data, analytics, Kubernetes heritage, and AI. The right choice is usually shaped by organizational skills, contracts, compliance, existing workloads, and service fit.

GitHub competes with Azure DevOps, GitLab, Bitbucket, and self-hosted Git systems. GitHub is widely used for open source and increasingly common in enterprise collaboration. Azure DevOps remains common in Microsoft enterprise shops with established Boards, Repos, and Pipelines usage.

CI/CD tools include GitHub Actions, Azure Pipelines, GitLab CI, Jenkins, TeamCity, CircleCI, Buildkite, and others. The best tool is the one your team can operate reliably with clear secrets management, repeatable builds, and useful feedback.

AI services compete across providers and hosting models: OpenAI, Azure OpenAI, Azure AI Foundry, GitHub Models, Amazon Bedrock, Google Gemini, local models through tools such as Ollama, and traditional ML through

ML.NET. Modern .NET's direction is to provide abstractions that let applications use model services without binding every part of the code to one provider.
The most common overengineering mistake is stacking all of these choices into the first version of an application. A strong architecture grows the stack when the system has earned the complexity.

Layer 7 — Interview Perspective

Interviewers use stack questions to test whether you understand boundaries, not whether you can recite logos.
Concepts commonly tested:

- the role of .NET versus ASP.NET Core;
- why Linux matters to modern .NET;
- the difference between containers and virtual machines;
- what Kubernetes adds on top of containers;
- why SQL Server and Redis are complementary rather than interchangeable;
- how GitHub, pull requests, and CI relate to team quality;
- what cloud platforms manage and what the team still owns;
- how AI services fit into an ordinary application architecture.

Representative questions:

- "Where does ASP.NET Core end and the hosting platform begin?"
- "Why would a .NET team use Docker?"
- "When would you avoid Kubernetes?"
- "What belongs in SQL Server versus Redis?"
- "How is GitHub Actions different from Git itself?"
- "What does a cloud provider manage for you, and what remains your responsibility?"
- "How would you add an AI feature without making the whole application depend on one model provider?"

A strong answer names the boundary and the tradeoff.
For example:

> "Redis can reduce load on SQL Server by caching frequently read data, but SQL Server remains the system of record. If Redis is unavailable, the application should either fall back to SQL Server or fail in a controlled way, depending on the feature."

Common misconceptions:

- ".NET is only a Windows platform."
- "ASP.NET Core requires IIS."
- "Docker is the same as Kubernetes."
- "Kubernetes makes applications cloud-native automatically."
- "Redis is just a faster database."
- "CI/CD is only a deployment script."
- "Cloud means no operations work."
- "AI services can be treated like deterministic libraries."

Small design scenario:
You are asked to design the first modern version of an internal order-tracking API. The company knows C#, SQL Server, and Windows Server. It wants to become more modern but does not yet have a large platform team.
A sensible stack might be:

- .NET and ASP.NET Core for the API;
- SQL Server for orders and durable business data;
- GitHub or Azure DevOps for source control and pull requests;
- CI to build and test every change;
- Docker for repeatable local dependencies and possibly packaging;
- a managed cloud app service or container app for hosting;
- basic logging, health checks, and metrics;
- Redis only if caching becomes necessary;
- Kubernetes only if the organization later needs orchestration at scale;
- AI services only when there is a concrete product feature, such as support note summarization or order-message classification.

The strong answer is not maximal. It is staged.

Hands-On Lab

Objective:
Create a stack map for a modern .NET application and classify each technology by responsibility.
Prerequisites:

- Chapter 1 completed.
- A text editor or notebook.
- Optional local installations of .NET, Git, Docker, and Kubernetes CLI tools.

Steps:

1. Draw five columns:

   ```
   Application | Data | Packaging/Runtime | Platform/Ops | Delivery
   ```

2. Place each technology from this chapter into the most natural column:

   ```
   .NET
   ASP.NET Core
   Linux
   Docker
   Kubernetes
   SQL Server
   Redis
   Cloud platform
   GitHub
   CI/CD
   AI services
   ```

3. For each technology, write one sentence explaining what problem it solves.
4. For each technology, write one sentence explaining what it does not solve.
5. Mark which technologies are essential for a first ASP.NET Core API.
6. Mark which technologies are optional until the system grows.
7. If you have the tools installed, run:

   ```
   dotnet --info
   git --version
   docker --version
   kubectl version --client
   ```

8. Rewrite the stack as a simple architecture diagram for an order-tracking API.

Expected results:

- You can describe the role of each major technology.
- You can separate application concerns from data, runtime, platform, and delivery concerns.
- You can identify which technologies are foundational and which are situational.

Validation commands:

```
dotnet --info
git --version
docker --version
kubectl version --client
```

Troubleshooting notes:

- If `docker` is not installed, complete the lab conceptually. Docker is taught later.
- If `kubectl` is not installed, that is fine. Kubernetes is intentionally not required yet.
- If you cannot decide where a technology belongs, ask what boundary it owns: code, data, runtime, platform, or delivery.

Knowledge Check

1. Why is ASP.NET Core not the same layer as Docker or Kubernetes?
2. What problem does Linux solve for modern .NET teams if Windows Server still exists?
3. Why is Docker useful even before an application runs in Kubernetes?
4. What does Kubernetes add beyond running a single container?
5. Why should SQL Server usually remain the system of record when Redis is added?
6. What is the difference between Git and GitHub?
7. Why is CI/CD part of the technology stack rather than only a team process?
8. What does a cloud platform manage, and what does it not manage for your application team?
9. Why should AI services be treated as runtime dependencies rather than ordinary deterministic libraries?
10. Which technologies in this chapter would you choose for the first version of a small internal API, and which would you postpone?

Summary

The modern .NET technology stack is a layered system.

.NET and ASP.NET Core provide the application foundation. SQL Server and Redis serve different data needs: durable relational state and fast shared in-memory state. Linux and Docker shape how applications run and how their environments are packaged. Kubernetes manages containerized workloads when scale and platform consistency justify its complexity. Cloud platforms provide managed compute, data, identity, networking, and operations services. GitHub and CI/CD turn source changes into validated, reviewable, repeatable delivery. AI services add model-backed behavior when the product needs capabilities that ordinary deterministic code does not provide.

The stack is not a checklist. It is a set of choices around boundaries. The professional skill is knowing which boundary you are addressing and whether the tool adds more clarity than complexity.

The next chapter moves from the technology map to the day-to-day workflow: how modern teams create, review, test, and move .NET changes through the software lifecycle.

Sources

- Microsoft Learn, ".NET releases, patches, and support": https://learn.microsoft.com/en-us/dotnet/core/releases-and-support
- Microsoft Learn, "ASP.NET documentation": https://learn.microsoft.com/en-us/aspnet/core/
- Docker Docs, "What is Docker?": https://docs.docker.com/get-started/docker-overview/
- Kubernetes Documentation, "Concepts": https://kubernetes.io/docs/concepts/
- Microsoft Learn, "What is SQL Server?": https://learn.microsoft.com/en-us/sql/sql-server/what-is-sql-server
- Redis, "What is Redis?: An Overview": https://redis.io/tutorials/what-is-redis/
- GitHub Docs, "GitHub Actions documentation": https://docs.github.com/en/actions
- Microsoft Learn, "Use continuous integration": https://learn.microsoft.com/en-us/devops/develop/what-is-continuous-integration
- Microsoft Azure, "What is cloud computing?": https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-cloud-computing/
- Microsoft Learn, "Develop .NET apps with AI features": https://learn.microsoft.com/en-us/dotnet/ai/overview
- DB-Engines, "DB-Engines Ranking": https://db-engines.com/en/ranking/
- CNCF, "2025 Annual Cloud Native Survey" announcement: https://www.cncf.io/announcements/2026/01/20/kubernetes-established-as-the-de-facto-operating-system-for-ai-as-production-use-hits-82-in-2025-cncf-annual-cloud-native-survey/
- Stack Overflow, "2025 Developer Survey: Technology": https://survey.stackoverflow.co/2025/technology/

Further Reading

- Microsoft Learn, ".NET + AI ecosystem tools and SDKs": https://learn.microsoft.com/en-us/dotnet/ai/dotnet-ai-ecosystem
- Google Cloud, "Google Cloud overview": https://docs.cloud.google.com/docs/overview
- AWS, "What is cloud computing?": https://docs.aws.amazon.com/whitepapers/latest/aws-overview/what-is-cloud-computing.html
- Docker Docs, "What is a container?": https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

CHAPTER 3

Modern Development Workflow

Chapter Purpose

The previous chapter named the major technologies in the modern .NET stack. This chapter shows how work moves through that stack during an ordinary professional development cycle.

If your baseline is Visual Studio 2019, Team Foundation Server or an older Git setup, manual QA handoffs, and deployments coordinated through people and servers, the modern workflow can look busier than the old one. There are pull requests, automated checks, cloud development environments, code review tools, pipeline statuses, package feeds, container images, security scans, and AI coding assistants.

The purpose of all that machinery is not ceremony. It is controlled feedback.

Modern development workflow exists because professional software teams learned the cost of late integration, unclear review, environment drift, manual deployment, and production surprises. The newer workflow tries to make each change small enough to understand, visible enough to review, automated enough to validate, and traceable enough to operate.

This chapter does not teach full CI/CD implementation. Chapter 8 covers automated testing and CI basics, and Chapter 16 covers deployment pipelines. Here, the goal is to understand the complete lifecycle at a high level: local development, source control, pull requests, testing, code reviews, AI coding assistants, development environments, and how code moves toward production.

Where This Fits

Chapter 1 shifted the mental model from server-centered development to platform-centered development. Chapter 2 identified the technologies in that platform. Chapter 3 connects them through time.

A modern workflow is a loop:

```
Plan a small change
        |
        v
Create or update code locally
        |
        v
Run local build and tests
        |
        v
Commit to a short-lived branch
        |
        v
Open a pull request
        |
        v
Review, discuss, and revise
        |
        v
```

```
Automated checks validate the change
        |
        v
Merge to the main branch
        |
        v
Build artifact or package
        |
        v
Deploy through a controlled process
        |
        v
Observe production behavior
        |
        v
Feed learning back into the next change
```

The key idea is that development does not end when code compiles. In a modern team, the workflow connects development, delivery, and operations. Microsoft describes DevOps as uniting people, process, and technology across planning, development, delivery, and operations. That is the lifecycle this chapter is mapping.
This workflow is now common across ecosystems, not only .NET. Java, Node.js, Python, Go, and .NET teams all use variants of this pattern because it solves a team coordination problem, not a language-specific problem.

Connection to the Reader's Existing Model

The familiar workflow probably had these landmarks:

- open the solution in Visual Studio;
- make a change;
- run locally with IIS Express or a local IIS site;
- maybe run unit tests;
- check in code;
- wait for a build or manually produce one;
- hand off to QA or operations;
- deploy by copying files, running a script, or using a release tool;
- troubleshoot on the server if something failed.

Modern development keeps the same broad intent but makes more of the workflow explicit and repeatable.
Visual Studio is still a powerful local environment, but it is no longer the only place where the project must build. A build should work from the command line, on another developer's machine, and on a clean build agent.
Source control is no longer just a backup and history mechanism. It is the coordination point for branches, pull requests, review, automated checks, security scanning, release notes, and sometimes infrastructure definitions.
A pull request is similar to a formal code review or change request, but it is attached directly to the code diff, tests, comments, approvals, and automation results.
Automated tests are similar to regression test scripts, but they execute as part of the normal change process instead of waiting for a separate late-stage test phase.
A CI check is similar to a build-server validation, but modern teams expect it to run frequently, consistently, and close to the code review.
A development environment is still a workstation-like place where you write and run code. The difference is that the environment may be local, container based, cloud hosted, or standardized through repository configuration.

An AI coding assistant is not a replacement for the developer. It is closer to an always-available pair programmer that can suggest code, explain APIs, draft tests, summarize changes, and help navigate unfamiliar areas. The analogy is useful as long as you remember that the assistant does not own correctness, security, or design judgment.
The old model often relied on expert memory: the senior developer knew how to build it, the release engineer knew how to deploy it, and the production admin knew which server setting mattered. The modern model tries to turn that memory into visible workflow.

Layer 1 — Conceptual Model

Modern development workflow is the repeatable path by which a team turns an idea into a safe, reviewed, validated, deployable software change.
It exists to solve five recurring problems.
First, integration delay. When developers work separately for too long, changes conflict, assumptions diverge, and bugs are discovered late. Short-lived branches, frequent commits, pull requests, and CI reduce the cost of bringing work together.
Second, environment drift. If every developer's machine, test server, and production server is slightly different, success in one place does not predict success in another. SDK-based builds, containerized dependencies, documented setup, and cloud development environments reduce drift.
Third, review ambiguity. A hallway conversation or email thread is easy to lose. Pull requests connect the code, conversation, tests, and approval record.
Fourth, manual validation. Humans are good at judgment and design review. Humans are poor at repeatedly remembering every mechanical check. Automated builds, tests, linters, formatters, and scans handle repeatable validation so people can focus on meaning.
Fifth, production disconnect. In older workflows, developers could finish a change and lose sight of it once it crossed into QA or operations. Modern workflow keeps feedback connected through deployment status, telemetry, incidents, and post-release learning.
What the workflow does not solve by itself:

- It does not guarantee good architecture.
- It does not make tests valuable just because they are automated.
- It does not replace code review judgment.
- It does not remove the need for production ownership.
- It does not make AI-generated code correct.
- It does not make a broken team healthy through tools alone.

The workflow is a support system for disciplined engineering. It makes good habits easier to repeat.

Layer 2 — System Relationships

The modern workflow has several connected actors and boundaries.
The developer owns the local change. Inputs include a work item, branch, existing code, local environment, tests, and context from previous chapters or documentation. Outputs include commits, tests, documentation updates, and a pull request.
The repository owns shared history. It receives commits and branches. It outputs version history, diffs, merge records, tags, and sometimes release artifacts. Its failure boundaries include bad branching practices, weak permissions, missing history, and unclear ownership.
The pull request owns review coordination. It receives a proposed diff, description, linked work item, review comments, automated check results, and updates from the author. It outputs an approved, rejected, or revised

change. Its failure boundaries include oversized changes, vague descriptions, rubber-stamp review, stale branches, and unresolved discussion.

The test suite owns repeatable validation. It receives compiled code, test data, dependencies, and environment configuration. It outputs pass/fail signals and diagnostics. Its failure boundaries include flaky tests, missing coverage for important behavior, over-mocked tests, slow feedback, and tests that assert implementation details instead of behavior.

The CI system owns clean validation. It receives repository events such as commits or pull requests. It checks out the code, restores dependencies, builds the solution, runs tests, and reports status. Its failure boundaries include broken agents, missing secrets, unavailable package feeds, nondeterministic tests, and mismatched SDK versions.

The code review process owns human judgment. It receives the author's intent, the diff, test evidence, design context, and automated results. It outputs feedback, suggestions, approval, or a request for changes. Its failure boundaries include reviewing style while missing behavior, deferring too much to automation, or treating review as personal criticism instead of shared ownership.

The development environment owns local productivity. It receives repository configuration, SDKs, tools, containers, secrets for local development, and IDE settings. It outputs fast feedback. Its failure boundaries include slow setup, hidden machine-specific assumptions, missing dependencies, and local-only success.

The AI coding assistant owns suggestion and acceleration, not authority. It receives prompts, visible code context, chat history, repository files, and possibly tool access depending on the environment. It outputs suggestions, explanations, edits, tests, or summaries. Its failure boundaries include plausible but wrong code, outdated assumptions, insecure suggestions, data exposure, and over-trust.

The deployment process owns movement toward runtime environments. In this chapter, deployment remains a high-level concept. Later chapters teach the mechanics. For now, understand that the output of development should be a validated change that can become a deployable artifact through a controlled path.

Layer 3 — Core Mechanics

The smallest useful workflow is a branch and pull request with automated validation.

```
main branch
  |
  +-- feature branch
        |
        v
      commits
        |
        v
      pull request
        |
        +-- human review
        |
        +-- automated build and tests
        |
        v
      merge
        |
        v
main branch with validated change
```

The main branch represents the shared integration line. Some teams call it `main`, some use `master`, and some use trunk-based development language. The name matters less than the rule: the shared line should be kept healthy.

A branch is an isolated line of work. Modern teams generally prefer short-lived branches because they reduce integration delay.
A commit is a recorded change. Good commits make the history understandable. They are not just save points; they are communication.
A pull request proposes that branch changes be merged into another branch. It contains the diff, discussion, review state, and automated checks.
A code review is a human evaluation of the proposed change. GitHub pull request reviews support comments, suggestions, approval, and requests for changes.
A check is an automated result attached to the proposed change. Checks might build the solution, run tests, enforce formatting, scan dependencies, or produce preview artifacts.
Continuous integration is the practice of automatically building and testing code when changes are committed to version control. It exists to catch integration problems early.
Continuous delivery is the broader practice of automating build, test, configuration, and deployment movement toward environments. This chapter mentions it only as part of the lifecycle. The detailed release pipeline comes later.
A development environment is the place where the developer writes and runs the application. It can be a local machine, a local container setup, a cloud environment such as GitHub Codespaces, or a managed developer workstation such as Microsoft Dev Box.
An AI coding assistant is a tool that uses large language models to assist with coding tasks. Examples include inline suggestions, chat-based code explanation, refactoring help, test generation, and debugging support. The state-of-the-art direction in 2026 is broader than autocomplete: assistants are increasingly integrated into IDEs, repositories, pull requests, terminals, cloud consoles, and agent-like workflows. That power makes review and judgment more important, not less.

Layer 4 — Developer Workflow

Here is the modern workflow in practical terms.
Start with a small piece of work. It might come from a backlog item, bug report, incident follow-up, architecture decision, or direct user need. The unit of work should be small enough that someone else can review it without reconstructing the entire system.
Update your local repository:

```
git status
git pull
```

Create a branch:

```
git switch -c feature/add-order-status
```

Run the application or tests before changing code when practical. This gives you a baseline:

```
dotnet build
dotnet test
```

Make the change in your editor or IDE. Visual Studio, Visual Studio Code, JetBrains Rider, Codespaces, and Dev Box can all participate in modern .NET workflows. The specific tool matters less than whether the workflow is repeatable outside one tool.
Use an AI coding assistant carefully if available. Good uses include asking it to explain unfamiliar code, draft a test shape, identify edge cases, summarize a diff, or suggest a refactoring. Risky uses include accepting large generated changes without understanding them, pasting secrets into prompts, or treating generated code as reviewed code.
Run local validation:

```
dotnet format --verify-no-changes
dotnet build
dotnet test
```

The exact commands vary by repository. Some teams use scripts, `make`, `just`, PowerShell, npm, Docker Compose, or custom build tools. The principle is that the repository should expose a known way to validate the change.
Commit the change:

```
git status
git add .
git commit -m "Add order status validation"
```

Push the branch:

```
git push -u origin feature/add-order-status
```

Open a pull request. The pull request description should explain what changed, why it changed, how it was tested, and any risk or follow-up work. If the change has screenshots, logs, migration notes, or API examples, include them.
Respond to review. Review is not a ceremony after the real work; it is part of the work. A reviewer may find a missed case, unclear name, fragile test, hidden security issue, or simpler design.
Wait for automated checks. If checks fail, fix the branch rather than hoping the failure is unrelated. If the failure is flaky, treat that as a workflow problem worth improving.
Merge after review and checks pass. Depending on team policy, the merge may create a merge commit, squash commits, or rebase. That policy is less important than keeping shared history understandable and the main branch healthy.
After merge, the change moves toward deployment. In a mature environment, the same change can be traced from work item to pull request, commit, build artifact, deployment, and production telemetry.

Layer 5 — Production Usage

Workflow choices affect production quality.
Configuration should be separated from code. Local development may use local settings or user secrets. CI may use test configuration. Production may use environment variables, managed secret stores, or platform configuration. Workflow should make it hard to accidentally commit secrets.
Security should shift earlier without pretending development can catch everything. Modern workflows often include dependency scanning, secret scanning, branch protection, least-privilege repository access, required reviews, and controlled deployment credentials. These practices reduce risk before the application reaches production.
Reliability starts before deployment. Small changes are easier to review, test, deploy, and roll back. Automated tests reduce regression risk. Pull requests capture design judgment. CI confirms that the change works outside the author's machine.
Deployment should consume validated artifacts. A common failure in older systems was rebuilding or modifying files during release in ways that made it unclear what actually reached production. Modern workflows try to build once, identify the artifact, and move that artifact through environments.
Observability closes the loop. Production logs, metrics, traces, errors, and alerts tell the team whether the change behaved as expected. Without feedback, deployment is a cliff.
Scaling affects workflow because larger teams need clearer boundaries. A three-person team can coordinate with conversation. A thirty-person team needs branch policies, ownership rules, review expectations, consistent local setup, and automation that prevents accidental damage.

Persistence affects workflow because database changes must be coordinated with application changes. A schema migration, data correction, or index change is not just "code." It has deployment ordering, rollback, locking, backup, and performance implications.
Cost affects workflow when builds, cloud development environments, preview environments, AI tools, and test infrastructure are metered. A modern workflow should give fast feedback without silently creating waste.
Local-development arrangements optimize for fast feedback. Production designs optimize for reliability, security, auditability, and recoverability. A good workflow keeps those goals connected without pretending they are identical.

Layer 6 — Tradeoffs and Alternatives

There is no single correct workflow for every team.
Small teams may use lightweight GitHub repositories, short branches, pull requests, and a few required checks. Larger enterprises may add issue tracking, change approval, security gates, compliance evidence, release trains, and environment promotion.
GitHub is widely used, especially for open source and many modern product teams. Azure DevOps remains common in Microsoft-oriented enterprises. GitLab, Bitbucket, Jenkins, TeamCity, CircleCI, and Buildkite are also valid choices. The workflow principles matter more than the brand: versioned code, review, automated validation, traceable delivery, and feedback from production.
Pull requests are not the only collaboration model. Some high-performing teams use trunk-based development with very short-lived branches and strong automation. Some regulated environments require heavier approval processes. Some open-source projects rely on maintainer review across forks. The best choice depends on team size, risk, deployment frequency, and compliance needs.
Local development is not always enough. A standard workstation may be fastest for many developers. A containerized setup may reduce dependency drift. GitHub Codespaces can provide cloud-hosted, repository-configured development environments. Microsoft Dev Box can provide secure, managed cloud development workstations. These options solve setup and consistency problems, but they add cost and administrative decisions.
AI coding assistants can accelerate exploration, routine code, test drafts, documentation, and unfamiliar API usage. They can also produce confident mistakes. Use them as collaborators under review, not as authorities.
Common overengineering mistakes:

- requiring a pull request for every tiny experimental spike;
- creating so many pipeline gates that developers stop trusting the system;
- running slow checks on every keystroke-level change;
- adding cloud development environments before local setup is understood;
- treating code review as permission bureaucracy instead of design feedback;
- accepting AI-generated code because it looks polished;
- confusing a sophisticated workflow with a healthy engineering culture.

Simpler alternatives are sometimes better. A small internal tool may need Git, a README, a build command, and a modest test suite before it needs preview environments and advanced deployment rings. More advanced alternatives are worthwhile when the risk and scale justify them: trunk-based development, ephemeral environments, progressive delivery, feature flags, policy-as-code, supply-chain signing, and platform-engineered paved paths.
The state-of-the-art direction is not heavier process. It is shorter feedback loops with stronger guardrails.

Layer 7 — Interview Perspective

Interviewers use workflow questions to learn how you work with a team.
Concepts commonly tested:

- the purpose of source control beyond backup;
- branch and pull request workflow;
- what belongs in a pull request description;
- the difference between local tests and CI checks;
- the purpose of code review;
- how to handle failed builds;
- how AI coding assistants should and should not be used;
- how a code change moves toward production.

Representative questions:

- “Walk me through your development workflow from task to merge.”
- “What makes a pull request easy to review?”
- “What should happen before code is merged to the main branch?”
- “How do you respond when CI fails but the code works locally?”
- “What is the difference between continuous integration and continuous delivery?”
- “How do you use AI coding tools responsibly?”
- “How would you improve a team that deploys manually and often has release surprises?”

A strong answer describes feedback loops. For example:

> “I try to keep changes small, run local validation before opening the pull request, explain the intent and test evidence, respond to review, and treat CI failures as real until understood. After merge, the change should be traceable through build and deployment so production feedback can be tied back to the code.”

Common misconceptions:

- “Source control is mainly a backup.”
- “A pull request is only for catching syntax mistakes.”
- “If it works locally, CI is optional.”
- “Code review is less important when tests pass.”
- “AI-generated code does not need the same review as human-written code.”
- “DevOps means developers do all operations work.”
- “Continuous delivery means every commit must go directly to production.”

Small design scenario:
You join a team maintaining a .NET line-of-business application. Developers work directly on a shared branch, releases are manual, tests are mostly run by QA near the end, and production fixes often happen under pressure.
A good modernization plan would be incremental:

- move to short-lived branches and pull requests;
- require a clear description and reviewer before merge;
- add a clean build check;
- add a small but meaningful automated test set;
- document local setup;
- protect the main branch;
- make release artifacts identifiable;
- improve production logging before attempting aggressive deployment automation;
- introduce AI coding tools with review and security expectations.

The answer should improve feedback before adding ceremony.

Hands-On Lab

Objective:
Practice the shape of a modern workflow using a small repository-level change.
Prerequisites:

- Git installed.
- .NET SDK installed.
- A local repository.
- No remote repository is required for the local parts of this lab.

Steps:

1. Inspect the repository state:

   ```
   git status
   ```

2. Create a branch:

   ```
   git switch -c workflow-practice
   ```

3. Inspect the installed .NET SDK:

   ```
   dotnet --info
   ```

4. If the repository has a solution or project, run:

   ```
   dotnet build
   dotnet test
   ```

   If the repository does not yet contain buildable code, write down the commands you expect the repository to support once code exists.

5. Create a short pull request draft in a text file or notebook with these headings:

   ```
   Summary
   Why
   Testing
   Risks
   Follow-up
   ```

6. Write a sample review comment against your own hypothetical change. Make it specific, respectful, and actionable.
7. Write one AI-assistant prompt that would be useful and safe for this work. For example:

   ```
   Review this small diff for edge cases and missing tests. Do not rewrite the
   implementation unless you find a specific issue.
   ```

8. Return to your original branch when finished:

   ```
   git switch -
   ```

Expected results:

- You can describe the workflow from branch to pull request to validation.
- You can distinguish local validation from CI validation.
- You can write a useful pull request description.
- You can state how AI assistance fits into review rather than replacing it.

Validation commands:

```
git status
git branch --show-current
dotnet --info
dotnet build
dotnet test
```

Troubleshooting notes:

- If `git switch -c` fails because the branch already exists, choose another branch name.
- If `dotnet build` fails because there is no project file, that is acceptable for this chapter. The next chapters begin building code.
- If `dotnet test` reports no test projects, note that as a future workflow gap rather than an error.
- If you use an AI assistant, do not paste secrets, private credentials, or confidential production data into the prompt.

Knowledge Check

1. Why is modern development workflow best understood as a feedback loop?
2. What problems do short-lived branches reduce?
3. What information should a pull request description provide to reviewers?
4. Why are automated checks useful even when reviewers are experienced?
5. How is continuous integration different from continuous delivery?
6. Why is "works on my machine" weaker evidence than a passing CI build?
7. What kinds of comments make code review more useful?
8. How can cloud development environments reduce onboarding friction?
9. What risks do AI coding assistants introduce into the workflow?
10. Why should production telemetry feed back into development planning?

Summary

Modern .NET development workflow is the path from idea to validated change to production feedback.

The core practices are not mysterious: work in small increments, keep source history clear, review proposed changes, automate repeatable validation, protect the shared branch, package changes consistently, and learn from production behavior. The tools have changed, but the goal is familiar: make software change safer.

Compared with older workstation-and-server workflows, the modern workflow places more responsibility in shared systems: Git repositories, pull requests, CI checks, documented development environments, controlled delivery, and observability. AI coding assistants now participate in that workflow, but they do not replace the developer's responsibility for correctness, security, and design.

The next chapter begins the practical .NET layer. With the workflow map in place, you can now look at modern .NET versions, SDKs, project structure, cross-platform development, and language improvements as parts of a larger professional system.

Sources

- Microsoft Learn, "What is DevOps?": https://learn.microsoft.com/en-us/devops/what-is-devops
- Microsoft Learn, "Use continuous integration": https://learn.microsoft.com/en-us/devops/develop/what-is-continuous-integration

- Microsoft Learn, “What is continuous delivery?”: https://learn.microsoft.com/en-us/devops/deliver/what-is-continuous-delivery
- GitHub Docs, “Quickstart for reviewing pull requests”: https://docs.github.com/en/pull-requests/get-started/reviewing-pull-requests-quickstart
- GitHub Docs, “Requesting a pull request review”: https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/requesting-a-pull-request-review
- GitHub Docs, “What are GitHub Codespaces?”: https://docs.github.com/en/codespaces/about-codespaces/what-are-codespaces
- Microsoft Learn, “Open a dev box in VS Code”: https://learn.microsoft.com/en-us/azure/dev-box/how-to-set-up-dev-tunnels
- Microsoft Learn, “GitHub Copilot Fundamentals”: https://learn.microsoft.com/en-us/training/paths/copilot/

Further Reading

- Microsoft Learn, “Deliver with DevOps”: https://learn.microsoft.com/en-us/training/modules/deliver-with-devops/
- GitHub Docs, “About pull requests”: https://docs.github.com/en/pull-requests/collaborating-with-pull-requests
- GitHub Docs, “GitHub Actions documentation”: https://docs.github.com/en/actions
- Microsoft Learn, “Adopt, extend and build Copilot experiences across the Microsoft Cloud”: https://learn.microsoft.com/en-us/microsoft-cloud/dev/copilot/overview

CHAPTER 4

Modern .NET

Chapter Purpose

The first three chapters established the modern .NET landscape, the technology stack, and the development workflow. This chapter moves into the platform at the center of that system: modern .NET itself.

For an experienced developer returning from the .NET 5 era, the important shift is not that C# suddenly became unfamiliar. The shift is that .NET is now a fast-moving, cross-platform, SDK-driven platform with a regular support cadence, simpler project files, stronger command-line workflows, first-class container and cloud integration, and ongoing language evolution.

Modern .NET exists because the old .NET world had reached a boundary. .NET Framework was powerful and deeply integrated with Windows, IIS, Visual Studio, and enterprise infrastructure, but that same integration made it hard to move at cloud speed. The industry was moving toward open source, Linux servers, containers, command-line automation, microservice-friendly runtimes, and frequent platform updates. Java, Node.js, Go, Python, and other ecosystems had already made cross-platform server development feel ordinary.

.NET Core began as Microsoft's answer to that reality: a modular, cross-platform, open-source version of .NET. .NET 5 unified the branding around "one .NET" for modern workloads. By 2026, modern .NET is no longer a side path. It is the main .NET line.

This chapter explains modern .NET versions, SDKs, project structure, cross-platform development, and language improvements. It gives enough practical mechanics to orient you without turning into a full API reference.

Where This Fits

Modern .NET is the application platform layer underneath the rest of the book.

```
Developer workflow
  |
  v
.NET SDK and project system
  |
  v
C# source code, project files, NuGet packages
  |
  v
Build, test, publish
  |
  v
Runtime process
  |
  +-- ASP.NET Core
  +-- worker services
  +-- console tools
  +-- libraries
  |
```

```
  v
Windows, Linux, containers, cloud platforms
```

Chapter 5 builds on this foundation with ASP.NET Core. Chapter 6 applies it to data access. Chapter 8 uses it for automated testing and CI. Chapters 9-13 use it across Linux, Docker, deployment, and cloud hosting.
The key point: modern .NET is both a runtime and a development platform. The runtime runs applications. The SDK builds, tests, publishes, packages, and tools them. The project system describes how source code becomes output.

Connection to the Reader's Existing Model

You already understand .NET as a managed platform: C# compiles to assemblies, the runtime executes managed code, libraries provide reusable APIs, and project files describe how the application is built.
That model still works, but several boundaries have moved.
In .NET Framework, the installed framework felt like part of the Windows machine. Applications targeted a framework version that was installed on the server. Visual Studio was often the center of build and publish behavior. Project files were verbose. IIS was the natural web host. Windows was assumed.
In modern .NET, the SDK is the center of development. The command line can create, build, test, run, publish, and package applications. Visual Studio, Visual Studio Code, Rider, GitHub Actions, Azure Pipelines, Docker builds, and cloud development environments all use the same underlying project and SDK model.
The modern project file is much smaller because SDK-style projects infer many defaults. A project file no longer needs to list every source file by default. NuGet package references are first-class. Target frameworks are explicit.
Side-by-side installation is normal. A machine can have several SDKs and runtimes installed. A repository can use `global.json` to select the SDK range used by command-line builds. This is very different from thinking of the server as having one blessed framework personality.
Cross-platform hosting is normal. A .NET application may be developed on Windows, built on Linux, tested in CI, packaged into a container, and deployed to a managed cloud environment. That does not mean Windows is obsolete. It means Windows is one supported environment rather than the assumption under every layer.
The IIS analogy remains useful for understanding process hosting, ports, configuration, and deployment. It breaks down when you assume IIS owns the application lifecycle. In modern .NET, an ASP.NET Core application is itself a process. IIS may reverse proxy to it, but so may Nginx, a cloud load balancer, a container platform, or a local development server.

Layer 1 — Conceptual Model

Modern .NET is the current, cross-platform .NET implementation for building applications, services, libraries, tools, cloud workloads, desktop apps, mobile apps, games, and AI-enabled systems.
It solves several problems:

- it provides a common managed runtime for C# and other .NET languages;
- it supports cross-platform development and hosting;
- it supplies a consistent SDK and command-line workflow;
- it uses a simpler project format;
- it supports side-by-side versions and regular releases;
- it integrates naturally with packages, tests, containers, cloud platforms, telemetry, and modern tooling.

It does not solve every application concern:

- it does not choose your architecture;
- it does not make your code maintainable automatically;

- it does not remove the need to understand deployment;
- it does not replace SQL Server, Redis, or cloud platforms;
- it does not guarantee cross-platform behavior if your code uses platform-specific APIs;
- it does not make every old .NET Framework application portable without work.

The conceptual center is this:

```
SDK describes how to build.
Project file describes what to build.
Target framework describes which APIs are available.
Runtime executes the built application.
```

Once that model is clear, most modern .NET mechanics become easier to place.

Layer 2 — System Relationships

Modern .NET interacts with the rest of the system through a few important relationships.
The SDK receives project files, source files, package references, target frameworks, analyzers, tool manifests, and build properties. It outputs compiled assemblies, test results, NuGet packages, publish folders, container images in some scenarios, and diagnostic information.
The runtime receives compiled assemblies, dependencies, configuration, command line arguments, environment variables, and operating-system services. It outputs process behavior: HTTP responses, console output, logs, files, network calls, database calls, and exit codes.
The project file owns build intent. It tells the SDK which project SDK to use, which target framework to compile against, which packages to reference, and which build options matter. In SDK-style projects, many conventions are implicit, which keeps the file small but makes it important to understand the defaults.
NuGet owns package distribution. A modern .NET project usually depends on NuGet packages from nuget.org, private feeds, or local package sources. Package restore is therefore part of build reliability and supply-chain security.
The operating system owns process execution, files, environment variables, networking, certificates, native dependencies, and permissions. If an application uses only cross-platform .NET APIs, it can often run on Windows, Linux, and macOS. If it depends on Windows-specific APIs, COM components, registry behavior, GDI, or old framework libraries, portability becomes a design constraint.
The CI system owns clean reproducibility. It uses the SDK to restore, build, test, and publish without relying on a developer's machine. If the build works only inside one person's IDE, the project is not yet modern in workflow terms.
The deployment platform owns runtime placement. It may run the app as a Windows service, IIS-hosted process, Linux systemd service, container, managed app service, serverless function, or orchestrated workload. .NET supports many of these targets, but each target changes configuration and operations.
Failure boundaries include mismatched SDK versions, unsupported target frameworks, missing runtimes, package restore failures, platform-specific code, native dependency issues, incorrect runtime identifiers, broken publish settings, and assuming local development behavior will match production.

Layer 3 — Core Mechanics

The modern .NET mechanics begin with versions.
Microsoft ships major .NET releases annually. Releases alternate between Long Term Support (LTS) and Standard Term Support (STS). As of July 22, 2026, Microsoft lists .NET 10 as an LTS release supported until November 2028, .NET 9 as an STS release supported until November 2026, and .NET 8 as an LTS release supported until November 2026.

That version choice matters. For production enterprise applications, an LTS release is often the conservative default. An STS release may be useful when a team wants newer platform capabilities and accepts a faster upgrade rhythm.
The next mechanic is the distinction between SDK and runtime.
The runtime runs .NET applications. The SDK creates and builds them. A server may need only the runtime if it runs framework-dependent applications. A developer machine and CI agent need the SDK.
The target framework tells the compiler which API surface the project uses. For modern .NET, this appears as a target framework moniker:

```
<TargetFramework>net10.0</TargetFramework>
```

A library can target multiple frameworks when it needs to support more than one consumer:

```
<TargetFrameworks>net8.0;net10.0</TargetFrameworks>
```

Multi-targeting is useful for libraries. It is usually unnecessary for a simple application where the team controls the runtime environment.
The project SDK appears at the top of an SDK-style project file:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

For a web application, the project uses the web SDK:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

`ImplicitUsings` reduces repetitive `using` directives for common namespaces. `Nullable` enables nullable reference type analysis, which helps identify possible null-related bugs at compile time. If you are returning from older C# habits, nullable reference types are one of the most important language-era changes to take seriously.
Runtime identifiers, or RIDs, identify target platforms such as `win-x64`, `linux-x64`, and `osx-arm64`. They matter when publishing for a specific platform or using packages with native assets.
`global.json` can select the SDK version used by CLI commands in a repository:

```
{
  "sdk": {
    "version": "10.0.100",
    "rollForward": "latestFeature"
  }
}
```

This does not choose the runtime your project targets. The target framework does that. `global.json` chooses the SDK used to build.
C# evolves with .NET. As of .NET 10, C# 14 is the latest C# release. It includes features such as extension members, null-conditional assignment, `field` backed properties, improved `Span<T>` conversions, and file-based app support.

You do not need to memorize every new feature immediately. The important production habit is to distinguish language convenience from design clarity.

Layer 4 — Developer Workflow

The modern .NET workflow is SDK-first. You can still use Visual Studio, and many enterprise developers should, but the project should also behave predictably from the command line.
Check your environment:

```
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

Create a console application:

```
dotnet new console -n ModernDotnetDemo
cd ModernDotnetDemo
dotnet run
```

Inspect the project file:

```
cat ModernDotnetDemo.csproj
```

On Windows PowerShell:

```
Get-Content .\ModernDotnetDemo.csproj
```

Build and run:

```
dotnet build
dotnet run
```

Create a solution and add the project:

```
cd ..
dotnet new sln -n ModernDotnetDemo
dotnet sln ModernDotnetDemo.sln add ModernDotnetDemo/ModernDotnetDemo.csproj
```

Create a local tool manifest when a repository needs repeatable local tools:

```
dotnet new tool-manifest
dotnet tool restore
```

Local tools matter because they let a repository define tool dependencies without requiring every developer to install the same global tool manually. Examples later in the book may include formatters, EF Core tools, or other project-specific commands.
Create a `global.json` when a repository needs predictable SDK selection:

```
dotnet new globaljson --sdk-version 10.0.100 --roll-forward latestFeature
```

The exact SDK version should match the version your team supports. In CI, being explicit avoids surprising builds when an agent image updates.
Publish the application:

```
dotnet publish -c Release
```

This produces output that can be run on a machine with an appropriate runtime or packaged into a deployment artifact. Later chapters cover ASP.NET Core, containers, and deployment in detail.

The workflow principle is simple: the same basic commands should work locally, in CI, and in deployment automation.

Layer 5 — Production Usage

Production .NET usage starts with support.

Choose a supported .NET version. For most enterprise production applications, that usually means choosing an LTS version unless a specific STS feature justifies the shorter support window. Keep applications patched with servicing updates because Microsoft support expects supported releases to be on supported servicing levels.

Separate SDK concerns from runtime concerns. Build agents need SDKs. Runtime hosts may need only runtimes, or no preinstalled runtime if you publish self-contained or run in a container image that includes the runtime. Do not assume the production machine has the same SDKs as a developer workstation.

Configuration belongs outside the compiled application. Modern .NET supports configuration from JSON files, environment variables, command-line arguments, secret stores, and platform providers. Local development can use local settings. Production should use mechanisms appropriate to the host platform.

Secrets should not be committed to source control. User secrets can be useful for local development. Production secrets should come from managed secret stores, environment injection, platform identity, or deployment systems.

Reliability depends on predictable builds and repeatable publishes. A production artifact should be identifiable. You should know which commit, target framework, SDK, package versions, and deployment settings produced it.

Observability starts in the application. Even before advanced telemetry, modern .NET applications should emit meaningful logs and expose health where appropriate. ASP.NET Core expands on this in the next chapter.

Cross-platform production requires discipline. File paths, case sensitivity, line endings, certificates, native libraries, time zones, globalization, process signals, and filesystem permissions can differ between Windows and Linux. Code that assumes Windows behavior may compile but still fail when hosted elsewhere.

Cost enters through runtime choices. Self-contained deployments can be larger. Containers introduce base-image and registry management. Cloud hosts charge for compute, memory, storage, networking, and sometimes build minutes. Newer features such as Native AOT can reduce startup time and footprint in some workloads, but they also introduce compatibility and diagnostic tradeoffs.

Local development optimizes for speed and convenience. Production optimizes for supportability, security, repeatability, and clear ownership.

Layer 6 — Tradeoffs and Alternatives

Modern .NET is a strong choice when a team values C#, mature tooling, enterprise libraries, high-performance server workloads, Microsoft ecosystem integration, and cross-platform deployment.

It competes with Java, Go, Node.js, Python, Rust, Kotlin, Ruby, PHP, and other ecosystems. Java remains a major enterprise platform with Spring. Go is common for small, fast cloud-native services and infrastructure tooling. Node.js is natural for JavaScript-centered teams. Python dominates many data and automation workflows. Rust is compelling when memory safety and systems-level control matter. Modern .NET's advantage is the combination of C#, runtime performance, enterprise fit, tooling, and platform breadth.

Use modern .NET for new .NET applications unless you have a specific reason not to. Use .NET Framework only when the application depends on technologies that require it, such as some legacy ASP.NET, WCF server scenarios, Windows-specific desktop dependencies, or older third-party libraries that cannot move.

Choose LTS for conservative production systems. Choose STS only when the team has an explicit upgrade plan and wants newer features sooner.

Use SDK-style projects for modern projects. Avoid manually expanding project files into old-style file lists unless a special build requirement demands it.

Use multi-targeting for reusable libraries that must support multiple consumers. Avoid multi-targeting applications without a clear reason; it increases testing and support work.
Use self-contained deployment when you need to carry the runtime with the app. Use framework-dependent deployment when the host environment manages the runtime. Use containers when repeatable packaging and runtime environment consistency matter.
Use Native AOT selectively. It can improve startup time and deployment size for some workloads, especially small services and command-line tools, but it can limit reflection-heavy libraries and change diagnostics.
Common overengineering mistakes:

- upgrading target frameworks without checking package and hosting support;
- treating every new C# feature as a reason to rewrite old code;
- multi-targeting applications unnecessarily;
- pinning SDK versions so tightly that servicing becomes painful;
- ignoring nullable warnings because the project still compiles;
- assuming cross-platform means every API behaves identically everywhere;
- choosing self-contained, containerized, trimmed, and AOT deployment all at once before understanding the tradeoffs.

The state-of-the-art direction in modern .NET is pragmatic platform breadth: fast runtime, minimal hosting, container support, cloud integration, better diagnostics, optional Native AOT, strong language evolution, and AI-friendly libraries. The skill is knowing which capabilities the application actually needs.

Layer 7 — Interview Perspective

Interviewers use modern .NET questions to test whether you understand the platform model, not just C# syntax.
Concepts commonly tested:

- .NET Framework versus .NET Core versus modern .NET;
- LTS versus STS releases;
- SDK versus runtime;
- target frameworks and target framework monikers;
- SDK-style project files;
- NuGet package references;
- `global.json`;
- framework-dependent versus self-contained deployment;
- cross-platform hosting;
- nullable reference types;
- when to use newer language features.

Representative questions:

- “What changed between .NET Framework and modern .NET?”
- “What is the difference between the .NET SDK and the runtime?”
- “Why would a project use `global.json`?”
- “What does `net10.0` mean?”
- “When would you choose an LTS release?”
- “What makes SDK-style project files different from older project files?”
- “What should you check before running a .NET application on Linux?”
- “When would self-contained deployment be useful?”

A strong answer connects versioning, build, and runtime:

> "The target framework controls which APIs the project compiles against. The SDK builds the project, and `global.json` can influence which SDK is selected for CLI commands. The runtime is what actually executes the application. In production, those choices affect support, deployment size, patching, and compatibility."

Common misconceptions:

- ".NET is just a renamed .NET Framework."
- "The SDK and runtime are the same thing."
- "`global.json` chooses the target framework."
- "If it compiles on Windows, it is automatically Linux-ready."
- "LTS means the application never needs patching."
- "New C# syntax is always a net improvement."
- "Self-contained deployment is always better."

Small design scenario:
You inherit a .NET 5 internal API. It builds only in Visual Studio, has no `global.json`, targets an out-of-support framework, and is deployed manually to a Windows server.
A good modernization plan would:

- move to a supported LTS target framework;
- verify package compatibility;
- make the build work through `dotnet build`;
- add or update `global.json` if the team needs SDK predictability;
- enable nullable analysis thoughtfully if it is not already enabled;
- define a repeatable publish command;
- confirm whether the app has Windows-specific assumptions;
- defer containers, cloud migration, and advanced deployment until the platform baseline is healthy.

The strong answer upgrades the foundation before changing the hosting model.

Hands-On Lab

Objective:
Create a small modern .NET console application and inspect the SDK-style project model.
Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- A terminal.
- A text editor or IDE.

Steps:

1. Create a working folder:

```
mkdir ModernDotnetLab
cd ModernDotnetLab
```

2. Check installed SDKs and runtimes:

```
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

3. Create a console app:

```
dotnet new console -n HelloModernDotnet
cd HelloModernDotnet
```

4. Open `HelloModernDotnet.csproj` and identify:
   - the project SDK;
   - the target framework;
   - whether nullable reference types are enabled;
   - whether implicit usings are enabled.
5. Run the app:

```
dotnet run
```

6. Build in Release configuration:

```
dotnet build -c Release
```

7. Publish the app:

```
dotnet publish -c Release
```

8. Create a solution one directory above the project:

```
cd ..
dotnet new sln -n ModernDotnetLab
dotnet sln ModernDotnetLab.sln add HelloModernDotnet/HelloModernDotnet.csproj
```

9. Create a `global.json`:

```
dotnet new globaljson --roll-forward latestFeature
```

10. Re-run:

```
dotnet --info
```

Expected results:

- You can identify the SDK and runtime versions installed.
- You can explain the project SDK and target framework in the project file.
- You can build, run, and publish from the command line.
- You can explain what `global.json` does and does not control.

Validation commands:

```
dotnet --info
dotnet build
dotnet run
dotnet publish -c Release
```

Troubleshooting notes:

- If `dotnet` is not recognized, install the .NET SDK and reopen the terminal.
- If `dotnet new console` targets a different framework than expected, inspect which SDK is installed and selected.
- If `global.json` causes an SDK error, the requested SDK may not be installed or the roll-forward setting may be too restrictive.
- If publish output is hard to find, inspect the `bin/Release` folder under the project.

Knowledge Check

1. Why did modern .NET move away from the Windows-centered assumptions of .NET Framework?
2. What is the difference between the .NET SDK and the .NET runtime?
3. Why does the target framework matter to both compilation and production support?
4. When is an LTS release usually preferable to an STS release?
5. What does an SDK-style project file hide through convention?
6. What does `global.json` control, and what does it not control?
7. Why can cross-platform development still fail when the code compiles?
8. When would multi-targeting be useful?
9. What production tradeoff does self-contained deployment make?
10. Why should nullable reference type warnings be treated as design feedback instead of noise?

Summary

Modern .NET is the main .NET platform for current professional development. It grew out of the need for a cross-platform, open-source, cloud-ready, SDK-driven successor to the Windows-centered .NET Framework world.
The central mechanics are straightforward once the layers are separated. The SDK builds and tools applications. The runtime executes them. The project file describes build intent. The target framework describes the API surface. NuGet packages provide dependencies. `global.json` can select the SDK used by command line builds. Runtime identifiers describe target platforms when platform- specific publishing or native assets matter.
Modern .NET's power is not only newer APIs. It is that the same project model can participate in local development, command-line builds, CI, containers, Linux hosting, cloud platforms, and production deployment.
The next chapter builds on this foundation with ASP.NET Core, where modern .NET becomes a web and API application platform.

Sources

- Microsoft Learn, ".NET releases, patches, and support": https://learn.microsoft.com/en-us/dotnet/core/releases-and-support
- Microsoft Learn, "What's new in .NET 10": https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview
- Microsoft Learn, ".NET project SDKs": https://learn.microsoft.com/en-us/dotnet/core/project-sdk/overview
- Microsoft Learn, "Target frameworks in SDK-style projects": https://learn.microsoft.com/en-us/dotnet/standard/frameworks
- Microsoft Learn, ".NET Runtime Identifier catalog": https://learn.microsoft.com/en-us/dotnet/core/rid-catalog
- Microsoft Learn, "global.json overview": https://learn.microsoft.com/en-us/dotnet/core/tools/global-json
- Microsoft Learn, "Install .NET on Windows": https://learn.microsoft.com/en-us/dotnet/core/install/windows
- Microsoft Learn, "What's new in C# 14": https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-14

Further Reading

- Microsoft Learn, ".NET CLI overview": https://learn.microsoft.com/en-us/dotnet/core/tools/
- Microsoft Learn, ".NET tools": https://learn.microsoft.com/en-us/dotnet/core/tools/global-tools

- Microsoft Learn, “C# language reference”: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/
- Microsoft Learn, “.NET application publishing overview”: https://learn.microsoft.com/en-us/dotnet/core/deploying/

CHAPTER 5

ASP.NET Core

Chapter Purpose

Modern .NET becomes most visible when it is used to build web applications and HTTP APIs. ASP.NET Core is the framework that makes that happen.

If your previous web development model was ASP.NET MVC or Web API hosted in IIS, ASP.NET Core will feel familiar in some places and very different in others. It still handles HTTP requests, routing, controllers, configuration, dependency injection, logging, authentication, and responses. But it is no longer built around System.Web, IIS, or a Windows-only hosting model.

ASP.NET Core exists because the classic ASP.NET stack had become too tightly coupled to Windows, IIS, and a large framework surface. The industry had moved toward lightweight HTTP services, Linux hosting, containers, reverse proxies, cloud platforms, and high-throughput APIs. Frameworks such as Node.js Express, Ruby on Rails, Spring Boot, Django, FastAPI, and Go's HTTP ecosystem shaped developer expectations around simple startup, explicit routing, middleware, and portable hosting.

ASP.NET Core was Microsoft's answer: a redesigned, cross-platform, high-performance web framework for modern .NET. By 2026, it is the standard web framework for current .NET applications.

This chapter introduces ASP.NET Core's place in the system and its core building blocks: Minimal APIs, controllers, dependency injection, configuration, middleware, and logging. Later chapters go deeper into data, security, testing, deployment, observability, and cloud hosting.

Where This Fits

ASP.NET Core sits between users or clients and the rest of the application system.

```
Browser, mobile app, service client, or integration partner
        |
        v
HTTP request
        |
        v
ASP.NET Core host
        |
        +-- Middleware pipeline
        |
        +-- Routing
        |
        +-- Minimal API endpoint or controller action
        |
        +-- Application services through dependency injection
        |
        +-- Configuration and logging
        |
        v
Database, cache, queue, file storage, external API, or AI service
```

This chapter is the first point where the book turns the platform concepts from Chapters 1-4 into a running server application. It prepares for Chapter 6 on data access, Chapter 7 on authentication and security, Chapter 8 on testing and CI, and later deployment chapters.

ASP.NET Core is not the whole application. It is the HTTP boundary. A good ASP.NET Core application still needs clear business logic, data access, validation, security, deployment, and observability. The framework gives the request-processing structure where those concerns meet.

Connection to the Reader's Existing Model

Classic ASP.NET applications were naturally understood through IIS.

IIS received the request, selected the site and application pool, loaded the application, applied modules, handed the request into ASP.NET, and wrote the response. Configuration often lived in `web.config`. The application pool identity affected file and database access. IIS logs and Windows Event Log were primary diagnostic tools.

ASP.NET Core changes the center.

An ASP.NET Core application is a .NET process. That process contains a web host, configuration, services, middleware, routes, endpoints, and logging. IIS can still be involved, especially on Windows, but it is often a reverse proxy or hosting integration layer rather than the conceptual owner of the application.

The analogy to IIS application pools is still useful for process isolation. The analogy breaks down because ASP.NET Core can run behind IIS, Nginx, Apache, a cloud load balancer, a container platform, or directly through Kestrel. Kestrel is ASP.NET Core's cross-platform web server.

HTTP modules and handlers from classic ASP.NET map conceptually to middleware and endpoints. Middleware handles cross-cutting request behavior such as exception handling, HTTPS redirection, static files, routing, authentication, authorization, compression, and custom request processing. Endpoints handle the application-specific request.

Controllers remain familiar if you used ASP.NET MVC or Web API. They group related actions into classes, use attributes for routing and behavior, and work well for larger APIs with conventions, filters, and model binding.

Minimal APIs are newer. They let you define HTTP endpoints directly in code with less ceremony. They are especially useful for small APIs, microservices, and services where endpoint behavior is easier to read inline or in small route groups.

Configuration files still matter, but configuration is broader than files. ASP.NET Core can read configuration from JSON files, environment variables, command-line arguments, user secrets, and external providers. This matches modern deployment where production settings often come from the hosting platform.

Dependency injection is built in. If you previously used a third-party IoC container or manually constructed services, ASP.NET Core makes service registration and constructor injection part of the default application model.

Logging is built in as a framework abstraction. Instead of writing only to a local file or Windows Event Log, the application can emit structured events that local tools, cloud platforms, or observability systems can collect.

Layer 1 — Conceptual Model

ASP.NET Core is a framework for building HTTP-based applications on modern .NET.

It solves these problems:

- accepting HTTP requests;
- routing requests to application code;
- binding request data to .NET types;
- producing HTTP responses;
- composing cross-cutting request behavior;
- managing application services through dependency injection;

- reading configuration from multiple sources;
- emitting logs and diagnostics;
- hosting on Windows, Linux, containers, and cloud platforms.

It does not solve these problems by itself:

- it does not design your domain model;
- it does not choose your database strategy;
- it does not make endpoints secure unless security is configured;
- it does not guarantee API usability;
- it does not replace tests;
- it does not remove deployment and operations concerns;
- it does not make distributed systems simple.

The conceptual center is the request pipeline.

```
Request -> Middleware -> Routing -> Endpoint -> Response
```

Middleware is software assembled into a pipeline. Each middleware component can work before the next component, work after the next component, pass the request forward, or stop the pipeline early. Microsoft calls this stopping behavior short-circuiting.
An endpoint is the application code selected to handle a request. In ASP.NET Core APIs, the endpoint is often a Minimal API handler or a controller action.
Services are application dependencies registered with the dependency injection container. Endpoints, controllers, middleware, and other services can request those dependencies through constructors or parameters.
Configuration provides values the application needs without hard-coding them: connection strings, feature flags, external service URLs, logging levels, security settings, and environment-specific behavior.
Logging provides a record of what the application is doing. In modern systems, logs are part of production visibility, not just debugging.

Layer 2 — System Relationships

ASP.NET Core interacts with several surrounding components.
The client sends an HTTP request. The client might be a browser, mobile app, JavaScript frontend, another service, a scheduled job, a webhook provider, or an integration partner. The request includes method, path, headers, body, query string, cookies, and connection information.
The host starts the application process. It loads configuration, sets up logging, registers services, builds the middleware pipeline, maps endpoints, and begins listening for requests. In the minimal hosting model, this startup code usually lives in `Program.cs`.
The web server receives HTTP traffic. Kestrel is the built-in cross-platform web server used by ASP.NET Core. In production, Kestrel is often placed behind a reverse proxy, load balancer, or platform ingress that handles internet edge concerns.
The middleware pipeline owns cross-cutting HTTP behavior. Inputs include the request and current application state. Outputs include a modified request, response, logs, authentication state, errors, or an early response.
Routing selects the endpoint. It compares the HTTP method and path with mapped routes. It may also consider route constraints and endpoint metadata.
Minimal API handlers or controller actions own request-specific application behavior. They should validate input, call application services, and return clear responses. They should not become dumping grounds for database and business logic.

Application services own business operations. They are registered with dependency injection and consumed by endpoints. This keeps HTTP details from swallowing the entire application design.

Configuration providers own environment-specific values. Local development may use `appsettings.Development.json` and user secrets. Production may use environment variables, secret stores, or cloud configuration services.

Logging providers own output destinations. The application writes through the ASP.NET Core logging abstraction. Providers send those events to console, debug output, files, cloud systems, or observability backends.

Failure boundaries include startup failure, missing configuration, dependency registration errors, middleware ordering mistakes, route conflicts, model binding failures, invalid input, unhandled exceptions, downstream service failures, thread-pool exhaustion, connection limits, and incomplete logging.

The most common learning mistake is treating ASP.NET Core as one object called "the web app." It is better understood as a host, pipeline, endpoint model, service container, configuration system, and logging system working together.

Layer 3 — Core Mechanics

The smallest useful ASP.NET Core application is a Minimal API.

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.MapGet("/", () => "Hello from ASP.NET Core");

app.Run();
```

This small program contains the basic shape.

`WebApplication.CreateBuilder(args)` creates the builder. The builder gathers configuration, logging, services, and host settings.

`builder.Services` is where application services are registered:

```
builder.Services.AddSingleton<Clock>();
```

`builder.Build()` creates the application.

`app.MapGet("/", ...)` maps an HTTP GET request for `/` to an endpoint handler.

`app.Run()` starts the application and blocks until shutdown.

Middleware is added before the application starts:

```
app.UseHttpsRedirection();

app.MapGet("/time", (Clock clock) =>
{
    return new { utc = clock.GetUtcNow() };
});
```

The order matters. Middleware runs in the order it is registered. A middleware component can affect everything after it.

A simple service might look like this:

```
public sealed class Clock
{
    public DateTimeOffset GetUtcNow() => DateTimeOffset.UtcNow;
}
```

ASP.NET Core can provide that service to the endpoint because it was registered with dependency injection.
Controllers use a class-based model:

```
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public sealed class TimeController : ControllerBase
{
    private readonly Clock _clock;

    public TimeController(Clock clock)
    {
        _clock = clock;
    }

    [HttpGet]
    public IActionResult Get()
    {
        return Ok(new { utc = _clock.GetUtcNow() });
    }
}
```

To use controllers, the app registers controller services and maps controller endpoints:

```
builder.Services.AddControllers();

var app = builder.Build();

app.MapControllers();

app.Run();
```

Minimal APIs and controllers are not enemies. They are two endpoint styles. Microsoft currently recommends Minimal APIs for new HTTP API projects because they provide a simplified, high-performance approach with minimal code and configuration. Controllers remain useful for larger APIs, MVC conventions, filters, and teams that prefer class-based organization.
Configuration is available through `builder.Configuration`:

```
var connectionString = builder.Configuration.GetConnectionString("Orders");
```

Logging is available through `ILogger<T>`:

```
public sealed class OrderReporter
{
    private readonly ILogger<OrderReporter> _logger;

    public OrderReporter(ILogger<OrderReporter> logger)
    {
        _logger = logger;
    }

    public void ReportCreated(int orderId)
    {
        _logger.LogInformation("Order {OrderId} was created", orderId);
```

```
    }
}
```

The `{OrderId}` placeholder is a structured logging property. This is more useful than building one long string because logging systems can preserve the property as searchable data.

Layer 4 — Developer Workflow

Create a new ASP.NET Core web API:

```
dotnet new webapi -n Orders.Api
cd Orders.Api
dotnet run
```

The template creates an ASP.NET Core project using the web SDK:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
```

Run the app and look for the listening URLs in the console output. Local development typically uses HTTPS and a development certificate.
Inspect the available project templates:

```
dotnet new list web
```

Build the project:

```
dotnet build
```

Run with automatic rebuild during development:

```
dotnet watch
```

`dotnet watch` is useful because web development often involves small edits and fast feedback.
Call an endpoint from another terminal. The exact URL and endpoint depend on the template and launch profile:

```
curl https://localhost:5001/weatherforecast
```

On Windows PowerShell, use:

```
Invoke-RestMethod https://localhost:5001/weatherforecast
```

If the template uses a different port, use the URL printed by `dotnet run`.
Add a small endpoint in `Program.cs`:

```
app.MapGet("/health/basic", () => Results.Ok(new { status = "ok" }));
```

Run again and call the endpoint:

```
curl https://localhost:5001/health/basic
```

The developer workflow is intentionally simple at this stage:

```
Create project -> run locally -> add endpoint -> call endpoint -> inspect logs
```

Later chapters add data access, authentication, tests, containers, deployment, and observability.

Layer 5 — Production Usage

Production ASP.NET Core begins with hosting clarity.

Kestrel is the ASP.NET Core web server. In production it may run behind IIS, Nginx, Apache, a cloud load balancer, or a container ingress. The outer server or platform often handles TLS termination, request forwarding, static edge concerns, and network exposure. The ASP.NET Core app handles application routing and behavior.

Configuration should be environment-specific. Local development can use `appsettings.Development.json` and user secrets. Production should use environment variables, managed configuration, secret stores, or platform settings. Never commit production secrets.

Security includes more than login. HTTPS, forwarded headers, CORS, authentication, authorization, anti-forgery for browser forms, secure cookies, rate limiting, input validation, dependency patching, and safe error handling all matter. Chapter 7 covers authentication and authorization in detail.

Reliability depends on startup behavior, health checks, timeouts, cancellation tokens, retry policy, graceful shutdown, and clear failure modes when dependencies are unavailable. An API that hangs under dependency failure is harder to operate than one that fails clearly.

Deployment should use a repeatable artifact. ASP.NET Core apps can be published as framework-dependent deployments, self-contained deployments, container images, or platform-specific packages. Chapter 12 covers deployment more fully.

Observability starts with logs. ASP.NET Core emits framework logs and lets the application emit structured logs through `ILogger<T>`. Production systems should also expose health and later add metrics and tracing. Chapter 14 covers observability basics.

Scaling requires statelessness where possible. A load-balanced ASP.NET Core API should not assume that all requests from one user reach the same process. Session-like data, cache data, background work, and file storage need explicit design.

Persistence belongs behind clear service and data-access boundaries. Do not put SQL queries, migrations, and transaction logic directly into endpoint handlers unless the application is deliberately tiny. Chapter 6 covers data access.

Cost is affected by hosting choice, memory use, request volume, logging volume, dependency calls, and scaling settings. An inefficient endpoint can cost money in cloud environments even if it seems harmless locally.

Local development optimizes for fast iteration and clear debugging. Production optimizes for secure configuration, predictable startup, repeatable deployment, visibility, reliability, and controlled scale.

Layer 6 — Tradeoffs and Alternatives

Use ASP.NET Core when you need to build HTTP APIs, web applications, server-rendered pages, real-time services, gRPC services, backend-for-frontend services, or cloud-hosted .NET workloads.

Do not use ASP.NET Core when a simpler process is enough. A scheduled job, command-line import, Windows service, worker service, or serverless function may be a better fit if there is no HTTP boundary.

Minimal APIs are a strong default for new HTTP APIs, especially when endpoints are small, routing is explicit, and the team values low ceremony.

Controllers are strong when the API benefits from class-based organization, filters, conventions, attribute-heavy behavior, shared base controller patterns, or a team already has a mature controller-based style.

Razor Pages, MVC views, and Blazor are alternatives for applications that render user interfaces rather than only HTTP APIs. They are part of the ASP.NET Core family, but this chapter focuses on APIs because later samples build from API foundations.

Competing web frameworks include Spring Boot for Java, Express and Fastify for Node.js, Django and FastAPI for Python, Ruby on Rails, Laravel for PHP, and Go HTTP frameworks. ASP.NET Core is especially strong for C# teams that need performance, mature tooling, enterprise integration, and cloud-ready APIs. Stack Overflow's 2025 Developer Survey lists ASP.NET Core among the web frameworks and technologies used by professional developers, though JavaScript and Python web ecosystems remain very common across the broader industry.

State-of-the-art ASP.NET Core development in 2026 emphasizes Minimal APIs, OpenAPI descriptions, built-in dependency injection, structured logging, configuration from the host environment, container-friendly hosting, health-oriented operations, and careful security defaults.
Common overengineering mistakes:

- creating controllers, services, repositories, factories, and wrappers before the endpoint has real complexity;
- putting all business logic directly in Minimal API lambdas;
- treating middleware order as incidental;
- using global mutable state for request-specific behavior;
- returning inconsistent error shapes from related endpoints;
- logging secrets or entire request bodies in production;
- assuming local HTTPS, ports, and configuration match production hosting.

The right choice is usually staged: start with clear endpoints, simple service boundaries, good configuration, useful logs, and tests. Add framework features when they solve an actual pressure.

Layer 7 — Interview Perspective

Interviewers use ASP.NET Core questions to test whether you understand the modern web application model.
Concepts commonly tested:

- Kestrel and hosting;
- middleware pipeline and ordering;
- routing and endpoints;
- Minimal APIs versus controllers;
- dependency injection lifetimes;
- configuration providers;
- logging and structured logging;
- environment-specific behavior;
- production concerns such as HTTPS, health checks, and reverse proxies.

Representative questions:

- “How is ASP.NET Core different from classic ASP.NET?”
- “What is middleware?”
- “Why does middleware order matter?”
- “When would you choose Minimal APIs versus controllers?”
- “How does dependency injection work in ASP.NET Core?”
- “Where should configuration come from in production?”
- “What is Kestrel?”
- “How would you structure a small API so business logic does not live entirely in endpoint handlers?”

A strong answer connects the request pipeline to application boundaries:

> “ASP.NET Core receives a request through the host and passes it through an ordered middleware pipeline. Routing selects an endpoint, such as a Minimal API handler or controller action. That endpoint should coordinate application services through dependency injection rather than contain all business and data-access logic directly.”

Common misconceptions:

- “ASP.NET Core requires IIS.”

- “Minimal APIs are only for toy applications.”
- “Controllers are obsolete.”
- “Middleware order rarely matters.”
- “Dependency injection is optional plumbing with no design impact.”
- “Logging is only for debugging locally.”
- “Configuration files are the only configuration source.”
- “A passing local request proves the API is production-ready.”

Small design scenario:
You need to build an internal order-status API. It has five endpoints, calls SQL Server, and will later require authentication.
A good first design might use Minimal APIs with route groups, a small application service for order status operations, configuration for the database connection, structured logging, and simple health reporting. It would avoid Kubernetes, custom middleware, caching, or complex architecture until the API has a real need.
If the API grows to dozens of endpoints with shared filters, complex request models, and strong team conventions around controller organization, controllers may become the more natural endpoint style.

Hands-On Lab

Objective:
Create a small ASP.NET Core API, add an endpoint, register a service, and call the endpoint locally.
Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- A terminal.
- A browser, `curl`, or PowerShell.

Steps:

1. Create the API:

```
dotnet new webapi -n Chapter05.Api
cd Chapter05.Api
```

2. Run the application:

```
dotnet run
```

3. Note the HTTPS URL printed in the console.
4. In another terminal, call the default endpoint. Adjust the URL and path to match the template output:

```
curl https://localhost:5001/weatherforecast
```

5. Stop the application.
6. Add this service class:

```
public sealed class SystemClock
{
    public DateTimeOffset GetUtcNow() => DateTimeOffset.UtcNow;
}
```

7. Register the service in `Program.cs` before `builder.Build()`:

```
builder.Services.AddSingleton<SystemClock>();
```

8. Add an endpoint after the app is built:

```
app.MapGet("/time", (SystemClock clock) =>
{
    return Results.Ok(new { utc = clock.GetUtcNow() });
});
```

9. Run the application again:

```
dotnet run
```

10. Call the new endpoint:

```
curl https://localhost:5001/time
```

11. On Windows PowerShell, use:

```
Invoke-RestMethod https://localhost:5001/time
```

Expected results:

- The API starts locally.
- The default endpoint responds.
- The `/time` endpoint returns a JSON response containing a UTC timestamp.
- The service is provided through dependency injection.
- The console shows ASP.NET Core logging output.

Validation commands:

```
dotnet build
dotnet run
curl https://localhost:5001/time
```

Troubleshooting notes:

- If the port is different, use the URL printed by `dotnet run`.
- If HTTPS fails locally, trust the development certificate or use the HTTP URL printed by the application.
- If dependency injection fails, confirm the service was registered before `builder.Build()`.
- If the endpoint returns 404, confirm the route string and that the app was restarted after editing.

Knowledge Check

1. Why is ASP.NET Core better understood as a request-processing framework than as “IIS for modern .NET”?
2. What role does Kestrel play?
3. What is middleware, and why does order matter?
4. How is a Minimal API endpoint different from a controller action?
5. When might controllers be a better fit than Minimal APIs?
6. Why should application services be separated from endpoint handlers as an API grows?
7. What kinds of values belong in configuration rather than code?
8. Why is structured logging more useful than string-only logging?
9. What production concerns appear before data access or authentication are added?
10. How does ASP.NET Core’s hosting model support Windows, Linux, containers, and cloud platforms?

Summary

ASP.NET Core is the modern .NET framework for HTTP applications. It replaced the old System.Web-centered mental model with a cross-platform host, Kestrel, an ordered middleware pipeline, endpoint routing, built-in dependency injection, flexible configuration, and integrated logging.

Minimal APIs provide a low-ceremony way to build HTTP APIs and are the modern default for many new API projects. Controllers remain valuable for larger APIs and teams that benefit from class-based organization and conventions.

The most important idea is the pipeline: requests enter the host, pass through middleware, route to endpoints, use services through dependency injection, and return responses while emitting logs and diagnostics.

The next chapter adds the data layer. With ASP.NET Core as the HTTP boundary, the application now needs durable state, queries, migrations, and performance discipline.

Sources

- ASP.NET documentation
- ASP.NET Core fundamentals overview
- Tutorial: Create a Minimal API with ASP.NET Core
- APIs overview
- ASP.NET Core Middleware
- Overview of OpenAPI support in ASP.NET Core API apps
- Stack Overflow 2025 Developer Survey: Technology

Further Reading

- Understand ASP.NET Core fundamentals
- Configure services with dependency injection in ASP.NET Core
- Customize ASP.NET Core behavior with middleware
- Create web APIs with ASP.NET Core

CHAPTER 6

Data Access

Chapter Purpose

Most business applications eventually become data applications.

Chapter 5 introduced ASP.NET Core as the HTTP boundary. That boundary is useful only when the application can read and change durable state: customers, orders, invoices, users, permissions, audit records, product catalogs, and workflow history. Chapter 6 explains how modern .NET applications usually reach that data.

For a returning Windows, SQL Server, and C# developer, this chapter begins from familiar ground. SQL Server is still a major enterprise database. T-SQL, indexes, transactions, constraints, query plans, backup, restore, and security still matter. Modern .NET did not make relational thinking obsolete.

What changed is the application side of the boundary.

Modern .NET applications usually do not treat database access as a few connection strings hidden in `web.config` and a set of ad hoc ADO.NET calls spread through pages or controllers. They tend to use explicit data-access libraries, dependency injection, environment-based configuration, migrations, async APIs, source-controlled schema changes, and production observability.

Entity Framework Core, or EF Core, is Microsoft's modern object-relational mapper for .NET. It exists to let developers work with database-backed data through .NET objects while still using a relational database underneath.

Dapper is a popular micro-ORM originally developed for Stack Overflow. It exists for teams that want to write SQL directly while avoiding repetitive mapping code.

This chapter introduces SQL Server, EF Core, Dapper, migrations, and performance from a modern .NET perspective. It does not try to teach database administration or every ORM feature. The goal is to help you place data access correctly in an ASP.NET Core application and understand the tradeoffs.

Where This Fits

Data access sits behind the HTTP layer and inside the application boundary.

```
HTTP client
   |
   v
ASP.NET Core endpoint or controller
   |
   v
Application service
   |
   v
Data access boundary
   |
   +-- EF Core DbContext
   |
   +-- Dapper query object
   |
   +-- raw ADO.NET when needed
```

```
   |
   v
SQL Server or another database
```

The endpoint should not need to know every table, join, transaction detail, or connection-management rule. It should coordinate request handling. The application service should express the business operation. The data-access layer should own the database conversation.

This chapter prepares for several later chapters. Chapter 7 adds security. Chapter 8 validates code with tests and CI. Chapter 11 uses Docker Compose to run multi-container development environments, including local SQL Server. Chapter 12 discusses deployment. Chapter 14 covers diagnostics. Chapter 15 adds caching and distributed application patterns.

Connection to the Reader's Existing Model

You already understand SQL Server as a durable system of record.

You know that tables, keys, constraints, indexes, transactions, stored procedures, views, and query plans are not implementation trivia. They are the shape of business data. You also know that a database is not just a file the application happens to use. It has its own lifecycle, permissions, backups, capacity, and operational failure modes.

That knowledge remains valuable.

The modern shift is how the application describes and reaches the database. Instead of connection strings in `web.config`, ASP.NET Core usually reads connection strings from configuration providers. In local development, that may be `appsettings.Development.json` or user secrets. In production, it should be a secret store, environment variable, or managed platform setting.

Instead of creating `SqlConnection` everywhere, modern applications usually centralize data access through dependency injection. EF Core uses a `DbContext` as the unit that tracks entities, builds queries, and saves changes. Dapper uses an open database connection and extension methods such as `QueryAsync` and `ExecuteAsync`.

Instead of manually remembering which schema changes were applied to which database, EF Core migrations can record schema evolution in source control. This does not remove the need for DBA judgment. It gives the team a repeatable artifact to review, test, script, and deploy.

The analogy to stored procedures is useful. Stored procedures centralize database behavior close to the data. EF Core centralizes database mapping and query generation close to the application model. Dapper centralizes explicit SQL close to application code. Each style chooses a different place for database knowledge to live.

The analogy breaks down when an ORM is treated as if the relational database disappeared. It did not. EF Core still produces SQL. SQL Server still executes queries. Indexes still determine performance. Transactions still define consistency. The database is still real.

Layer 1 — Conceptual Model

Data access is the boundary between application behavior and durable or shared state.

It solves these problems:

- connecting to a database;
- translating application operations into database operations;
- querying data efficiently;
- saving changes consistently;
- managing transactions;
- evolving schema over time;
- keeping data-access details out of HTTP endpoints;

- making database behavior testable and observable.

It does not solve these problems by itself:

- it does not design a correct data model;
- it does not make slow queries fast automatically;
- it does not remove the need for indexes;
- it does not make distributed transactions simple;
- it does not protect secrets unless configuration is handled correctly;
- it does not replace backup, restore, monitoring, or operational planning.

The main modern .NET data-access choices are:

```
EF Core
  Higher-level object-relational mapping
  Strong integration with LINQ, DbContext, migrations, and change tracking

Dapper
  SQL-first micro-ORM
  Strong control over SQL with lightweight object mapping

ADO.NET
  Lowest-level database API
  Maximum explicit control with more repetitive code
```

EF Core is often the default starting point for modern .NET applications because it integrates well with .NET, ASP.NET Core dependency injection, LINQ, migrations, and multiple database providers.
Dapper is often chosen when SQL control is more important than ORM modeling, when queries are highly tuned, or when a team already thinks naturally in SQL.
ADO.NET remains underneath both approaches and is still available when direct control matters.

Layer 2 — System Relationships

The application sends data-access intent. It asks to create an order, load a customer, check a permission, search products, or record an audit event.
The data-access code turns that intent into database commands. With EF Core, LINQ queries and tracked entity changes become SQL commands. With Dapper, explicit SQL text and parameters become commands. With ADO.NET, the developer builds the command directly.
The database provider owns the database-specific connection and translation details. EF Core supports multiple providers, including SQL Server, SQLite, PostgreSQL, MySQL, and others. Dapper works through ADO.NET connections, so it can work with many databases that expose .NET data providers.
The database owns durable state, constraints, query execution, locking, transactions, indexes, storage, backup, and recovery.
The configuration system owns connection information. The application should not hard-code production connection strings. The connection string is an input to the application, not part of the compiled binary.
The dependency injection container owns service lifetimes. In ASP.NET Core, EF Core `DbContext` instances are commonly registered with a scoped lifetime, meaning one context instance is used for a request scope. Dapper-based designs often register a connection factory or data-access service rather than a single shared connection.
The migration history owns schema evolution. EF Core migrations generate C# files that describe schema changes and a model snapshot used to detect later changes. Those files should be reviewed and committed like other source files.

Failure boundaries include connection failures, authentication failures, missing permissions, deadlocks, blocking, timeouts, migration mistakes, transaction conflicts, inefficient queries, model/schema drift, missing indexes, connection pool exhaustion, and serialization problems when data is returned through an API.
One practical rule helps: the database is a dependency, but it is not a minor dependency. It is often the most important production boundary in the system.

Layer 3 — Core Mechanics

Start with a small entity:

```
public sealed class Order
{
    public int Id { get; set; }
    public string CustomerName { get; set; } = "";
    public decimal Total { get; set; }
    public DateTimeOffset CreatedAt { get; set; }
}
```

In EF Core, a `DbContext` represents a session with the database:

```
using Microsoft.EntityFrameworkCore;

public sealed class OrdersDbContext : DbContext
{
    public OrdersDbContext(DbContextOptions<OrdersDbContext> options)
        : base(options)
    {
    }

    public DbSet<Order> Orders => Set<Order>();
}
```

The ASP.NET Core application registers the context:

```
builder.Services.AddDbContext<OrdersDbContext>(options =>
{
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("Orders"));
});
```

An endpoint or service can then query:

```
app.MapGet("/orders/{id:int}", async (
    int id,
    OrdersDbContext db) =>
{
    var order = await db.Orders.FindAsync(id);

    return order is null
        ? Results.NotFound()
        : Results.Ok(order);
});
```

For creation:

```
app.MapPost("/orders", async (
    CreateOrderRequest request,
    OrdersDbContext db) =>
{
    var order = new Order
    {
        CustomerName = request.CustomerName,
        Total = request.Total,
        CreatedAt = DateTimeOffset.UtcNow
    };

    db.Orders.Add(order);
    await db.SaveChangesAsync();

    return Results.Created($"/orders/{order.Id}", order);
});

public sealed record CreateOrderRequest(
    string CustomerName,
    decimal Total);
```

This is useful for learning, but production code often moves business rules out of the endpoint and into application services.

With Dapper, the SQL is explicit:

```
using Dapper;
using Microsoft.Data.SqlClient;

public sealed class OrderQueries
{
    private readonly string _connectionString;

    public OrderQueries(IConfiguration configuration)
    {
        _connectionString =
            configuration.GetConnectionString("Orders")
            ?? throw new InvalidOperationException(
                "Missing Orders connection string.");
    }

    public async Task<Order?> GetOrderAsync(int id)
    {
        const string sql = """
            select Id, CustomerName, Total, CreatedAt
            from dbo.Orders
            where Id = @Id;
            """;

        await using var connection =
            new SqlConnection(_connectionString);

        return await connection.QuerySingleOrDefaultAsync<Order>(
            sql,
            new { Id = id });
```

```
    }
}
```

Notice the tradeoff. EF Core hides most SQL generation and gives change tracking, LINQ, and migrations. Dapper keeps SQL visible and gives lightweight mapping.
Migrations describe schema changes:

```
dotnet tool install --global dotnet-ef
dotnet ef migrations add InitialCreate
dotnet ef database update
```

For production, do not blindly apply migrations because a command exists. Microsoft recommends generating SQL scripts for production deployment so they can be reviewed and adjusted before being applied.

```
dotnet ef migrations script -o migrations.sql
```

Performance begins with query awareness. EF Core LINQ is not magic; it is translated to SQL. Dapper SQL is not automatically fast; it still needs good indexes, parameters, and query plans. The database engine always has the final say.

Layer 4 — Developer Workflow

A typical EF Core workflow in an ASP.NET Core API looks like this:

```
Define entity and DbContext
        |
        v
Add provider package
        |
        v
Configure connection string
        |
        v
Register DbContext with dependency injection
        |
        v
Create migration
        |
        v
Apply migration to local database
        |
        v
Write endpoint or service
        |
        v
Run and test
```

Create a web API:

```
dotnet new webapi -n Orders.Api
cd Orders.Api
```

Add EF Core packages for SQL Server:

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Install or restore EF Core tools:

```
dotnet new tool-manifest
dotnet tool install dotnet-ef
dotnet tool restore
```

Create a migration:

```
dotnet ef migrations add InitialCreate
```

Apply it to a local development database:

```
dotnet ef database update
```

List migrations:

```
dotnet ef migrations list
```

Check whether model changes are missing a migration:

```
dotnet ef migrations has-pending-model-changes
```

For Dapper:

```
dotnet add package Dapper
dotnet add package Microsoft.Data.SqlClient
```

Dapper usually does not manage schema. Teams using Dapper often manage schema with SQL scripts, a migration tool, database projects, Flyway, Liquibase, DbUp, RoundhousE, or another database-change process.

Local development may use SQL Server installed directly, SQL Server in a container, LocalDB on Windows, or a shared development instance. Chapter 11 returns to local multi-container environments.

The important workflow habit is that schema changes, code changes, and tests move together. A pull request that changes data shape should include the application change, migration or SQL script, test evidence, and deployment notes.

Layer 5 — Production Usage

Production data access is where small shortcuts become expensive.

Configuration should keep connection strings out of source control. Local development can use user secrets or local settings. Production should use a secret store, environment variable, managed identity pattern, or hosting platform configuration.

Security includes database authentication, least-privilege database users, encrypted connections, parameterized queries, secret rotation, auditing, and careful handling of personally identifiable or regulated data. Dapper and EF Core both support parameterized commands; string-concatenated SQL remains a security risk.

Reliability depends on timeouts, retries, transaction boundaries, connection pooling, idempotent operations where possible, and clear behavior when the database is unavailable. Retrying every failure is not wisdom. Some failures are transient. Some indicate bad data, deadlocks, or broken design.

Deployment must respect schema and application compatibility. A database migration can break older application versions if columns are renamed, constraints are changed, or data is transformed in one step. Safer production changes often use expand-and-contract patterns: add compatible schema, deploy application changes, backfill data, then remove old schema later.

Observability should include slow queries, command failures, connection pool pressure, timeout rates, migration history, and database health. Application logs should contain enough context to diagnose failures without logging secrets or sensitive data.

Scaling data access is often harder than scaling web servers. Adding more API instances can increase database load. Caching, read replicas, query tuning, pagination, background processing, and data partitioning can help, but each adds tradeoffs.

Persistence is the point of the database. Do not treat cache or in-memory data as durable. SQL Server remains the system of record unless the architecture explicitly defines another durable store.

Cost is affected by database tier, storage, backup retention, DTUs or vCores, IO, memory, licensing, managed-service choices, inefficient queries, and over-eager logging. A slow query can become a cloud bill.

Local development optimizes for convenience and reset capability. Production optimizes for durability, backup, auditability, security, tested schema changes, and predictable performance.

Layer 6 — Tradeoffs and Alternatives

EF Core is a strong default when the application has a rich domain model, needs LINQ queries, benefits from change tracking, wants migrations, and fits well with a model-first or code-first workflow.

Dapper is strong when SQL control is central, queries are hand-tuned, stored procedures are important, the data model is close to result shapes, or the team wants minimal abstraction over SQL.

ADO.NET is appropriate when maximum control matters, when avoiding extra dependencies is important, or when building infrastructure-level libraries. It requires more repetitive code.

Stored procedures remain valid. They can centralize data rules, support security patterns, improve plan stability in some cases, and align with DBA-led database ownership. They can also hide behavior from application source control if not managed carefully.

SQL Server competes with PostgreSQL, MySQL, Oracle, SQLite, MariaDB, cloud relational databases, document databases, key-value stores, and analytical systems. According to DB-Engines in July 2026, Microsoft SQL Server ranked third among relational database management systems, behind Oracle and MySQL and just ahead of PostgreSQL. That ranking does not decide your architecture, but it confirms SQL Server remains a major professional database.

PostgreSQL is a major open-source alternative and is especially prominent in cloud-native and open-source-heavy systems. SQLite is excellent for embedded, local, test, and single-file scenarios. Document databases fit data that is naturally document-shaped. Analytical stores fit reporting and large-scale analysis. Redis fits fast in-memory state, not relational truth.

Common overengineering mistakes:

- creating a repository abstraction over EF Core that adds no useful boundary;
- putting all queries directly in controllers or Minimal API handlers;
- treating migrations as production-safe without review;
- ignoring generated SQL because the LINQ looks clean;
- loading entire tables and filtering in memory;
- using lazy loading without understanding query volume;
- adding Redis before fixing obvious query and index problems;
- treating database changes as less important than C# changes.

State-of-the-art .NET data access in 2026 is pragmatic: use EF Core where its modeling and migrations help, use Dapper where explicit SQL is clearer, review schema changes, observe query behavior, and respect the database as a production system.

Layer 7 — Interview Perspective

Interviewers use data-access questions to test whether you understand both .NET and databases.

Concepts commonly tested:

- EF Core `DbContext`;
- entity classes and `DbSet<T>`;
- LINQ-to-SQL translation;
- change tracking;
- migrations;
- Dapper versus EF Core;
- parameterized queries;
- connection strings and secrets;
- transactions;
- indexes and query performance;
- avoiding business logic in controllers.

Representative questions:

- “What problem does EF Core solve?”
- “When would you choose Dapper instead of EF Core?”
- “What is a migration?”
- “Why should generated migration SQL be reviewed before production?”
- “What is the risk of lazy loading?”
- “How do you avoid SQL injection?”
- “What should not go in an ASP.NET Core endpoint handler?”
- “Why can adding more web servers make database performance worse?”

A strong answer keeps the database visible:

> “EF Core lets us work with entities and LINQ, but it still generates SQL that the database must execute. I use it where change tracking and migrations help, inspect generated SQL for important paths, use indexes and pagination, and review migrations before production.”

Common misconceptions:

- “EF Core means I do not need to understand SQL.”
- “Dapper is always faster, therefore always better.”
- “Migrations are just development convenience.”
- “A repository layer automatically makes data access clean.”
- “Redis can replace SQL Server for ordinary business data.”
- “Async database calls make slow queries fast.”
- “If an endpoint is simple, data access can stay in the controller forever.”

Small design scenario:
You are building an order API. The first version needs to create orders, read orders by ID, list recent orders, and update order status.
A good design might use SQL Server as the system of record, EF Core for ordinary CRUD and migrations, application services for business operations, and carefully indexed queries for common reads. If a reporting endpoint later needs a tuned join, Dapper can be introduced for that query without replacing EF Core everywhere.
The strong answer chooses tools per boundary instead of turning one data-access style into a religion.

Hands-On Lab

Objective:

Add EF Core data access to a small ASP.NET Core API and understand the shape of a migration.
Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- A SQL Server development database, SQL Server container, LocalDB on Windows, or another SQL Server-compatible development option.
- Basic familiarity with Chapter 5's ASP.NET Core API.

Steps:

1. Create a new API:

```
dotnet new webapi -n Chapter06.Api
cd Chapter06.Api
```

2. Add EF Core packages:

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Design
```

3. Create a local tool manifest and install EF Core tools:

```
dotnet new tool-manifest
dotnet tool install dotnet-ef
```

4. Add an `Order` entity and `OrdersDbContext`.
5. Add a development connection string named `Orders`.
6. Register the context:

```
builder.Services.AddDbContext<OrdersDbContext>(options =>
{
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("Orders"));
});
```

7. Add a migration:

```
dotnet ef migrations add InitialCreate
```

8. Inspect the generated migration files before applying them.
9. Apply the migration to the local database:

```
dotnet ef database update
```

10. Add one endpoint to create an order and one endpoint to retrieve an order.
11. Build and run:

```
dotnet build
dotnet run
```

Expected results:

- The project builds.
- EF Core packages are referenced.
- A migration is generated and visible in source.
- The local database schema is created or updated.
- The API can save and retrieve an order.

Validation commands:

```
dotnet build
dotnet ef migrations list
dotnet ef migrations has-pending-model-changes
dotnet ef database update
dotnet run
```

Troubleshooting notes:

- If `dotnet ef` is not found, run `dotnet tool restore`.
- If the connection fails, verify the connection string and SQL Server instance.
- If the migration cannot be created, confirm the `DbContext` is registered and the design package is referenced.
- If `has-pending-model-changes` reports changes, create a migration or review whether the model changed unintentionally.
- If the API runs but database calls fail, check database permissions and whether the migration was applied.

Knowledge Check

1. Why is data access a boundary rather than just a utility function?
2. What does EF Core's `DbContext` represent?
3. Why does EF Core not remove the need to understand SQL Server?
4. When is Dapper a better fit than EF Core?
5. What problem do migrations solve?
6. Why should migrations be reviewed before production deployment?
7. What is the risk of putting database code directly in endpoint handlers?
8. Why can adding more application instances increase database pressure?
9. What is the difference between durable state and cached state?
10. How should connection strings and secrets be handled differently in local development and production?

Summary

Modern .NET data access begins with the same truth experienced SQL Server developers already know: the database is a durable, operationally important system. Tables, indexes, transactions, constraints, and query plans still matter. EF Core adds a modern object-relational mapping layer with `DbContext`, entities, LINQ, change tracking, providers, and migrations. Dapper offers a lighter SQL-first path for teams that want explicit queries and simple object mapping. ADO.NET remains available underneath both.

The modern practice is not to hide the database. It is to create a clear, testable, observable boundary between application behavior and persistent state. Schema changes should be source-controlled, reviewed, tested, and deployed deliberately. Query performance should be measured against the real database, not assumed from clean C#.

The next chapter adds authentication and security, because once an API can read and write data, the next question is who is allowed to do so.

Sources

- What is SQL Server?
- Overview of Entity Framework Core
- Managing Database Schemas
- Migrations Overview

- Managing Migrations
- Applying Migrations
- Efficient Querying
- What's New in EF Core 10
- Dapper GitHub repository
- DB-Engines Ranking of Relational DBMS

Further Reading

- Entity Framework Core documentation
- SQL Server documentation
- Dapper documentation repository
- EF Core performance documentation
- Connection strings and configuration in ASP.NET Core

CHAPTER 7

Authentication and Security

Chapter Purpose

Chapter 6 gave the application durable state. Once an API can read and change business data, the next question is unavoidable: who is allowed to do it?

Authentication and security exist because useful systems have boundaries. Some callers are anonymous. Some are employees. Some are customers. Some are services. Some can read. Some can write. Some can approve payments, reset passwords, view audit records, or administer tenants. The application must know the difference.

For a developer coming from Windows, IIS, SQL Server, and enterprise application development, the familiar model may involve Windows Authentication, Active Directory groups, IIS settings, SQL logins, service accounts, and connection strings protected on a server. Those ideas still matter, but modern .NET applications commonly run behind reverse proxies, in containers, on cloud platforms, and across identity providers. Security is no longer only a server setting. It is an application, platform, identity, deployment, and operations concern.

ASP.NET Core provides authentication and authorization building blocks. OAuth 2.0 and OpenID Connect provide common protocol patterns. JSON Web Tokens, or JWTs, are commonly used to carry identity and authorization claims to APIs. Secrets management keeps credentials and sensitive configuration out of source control and ordinary application files.

This chapter introduces authentication, authorization, JWT, OAuth, OpenID Connect, and secrets management at the level needed for modern .NET API development. It does not make you an identity specialist. It gives you the map, terminology, and basic mechanics so later chapters can build securely.

Where This Fits

Security surrounds the HTTP and data boundaries introduced in Chapters 5 and 6.

```
Client
  |
  | obtains token or session through identity system
  v
ASP.NET Core API
  |
  +-- Authentication: who is the caller?
  |
  +-- Authorization: what may this caller do?
  |
  +-- Application service
  |
  +-- Data access
  |
  v
SQL Server, cache, queue, external API, or AI service
```

Security also touches deployment:

```
Source control -> CI/CD -> runtime platform
      |             |             |
      v             v             v
  no secrets     protected     managed identity,
  in code        variables     secret store,
                               TLS, audit
```

The application cannot treat authentication as a decorative endpoint attribute added at the end. Authentication affects request handling. Authorization affects business rules. Secrets affect configuration. Identity affects database, cloud, queue, and external API access.

Connection to the Reader's Existing Model

Windows enterprise systems often used a perimeter-oriented security model.
The application ran on a known Windows server. IIS might require Windows Authentication. Users belonged to Active Directory groups. The application pool identity accessed files or SQL Server. Connection strings lived in configuration files protected by server permissions. The network boundary did some of the work.
Modern systems use many of the same ideas with different shapes.
An application pool identity maps conceptually to a service identity or managed identity. It answers, "What is this running application allowed to access?"
An Active Directory group maps conceptually to claims, roles, and policies. It answers, "What does this authenticated user or service represent?"
IIS authentication settings map conceptually to ASP.NET Core authentication schemes and middleware. The request still needs an identity before protected resources can be accessed, but the mechanism may be cookies, JWT bearer tokens, OpenID Connect, certificates, API keys, or platform-provided identity.
SQL Server permissions still matter. Application authorization should not be the only protection around data. The database account or managed identity should have only the access the application needs.
The analogy breaks down when the application is no longer protected mainly by a corporate network or one IIS server. Public APIs, mobile clients, cloud services, service-to-service calls, and external identity providers require explicit trust decisions. "It is inside the network" is not a complete security model.

Layer 1 — Conceptual Model

Authentication determines identity.
Authorization determines access.
Secrets management protects sensitive values the application needs to operate.
Those three concepts are related, but they are not interchangeable.
Authentication answers:

```
Who is calling?
```

Authorization answers:

```
What is this caller allowed to do?
```

Secrets management answers:

```
How does the application safely obtain credentials and sensitive settings?
```

Authentication solves the problem of establishing a caller's identity. In an ASP.NET Core API, the result is usually a `ClaimsPrincipal` : an identity plus claims about that identity.

Authorization solves the problem of making access decisions. It can use roles, claims, policies, resource ownership, tenant boundaries, business state, or a combination of those.
JWT bearer authentication solves a common API problem: a client can send a token in the `Authorization` header, and the API can validate that token without storing server-side session state for every request.
OAuth 2.0 solves delegated authorization. It lets a client obtain access to a resource with scoped permission, often without giving the client the user's password.
OpenID Connect builds on OAuth 2.0 to provide authentication and sign-in. It adds ID tokens and standardized identity information.
Secrets management solves the problem of keeping passwords, connection strings, API keys, certificates, signing keys, and client secrets out of code, logs, and casual configuration files.
Security does not solve these problems automatically:

- it does not make poor authorization rules correct;
- it does not protect secrets that are logged or committed;
- it does not make JWT contents trustworthy unless the token is validated;
- it does not replace input validation;
- it does not make production safe without patching and monitoring;
- it does not remove the need for least privilege.

Layer 2 — System Relationships

The client initiates a request. It may be a browser, mobile app, JavaScript frontend, server-side app, scheduled job, webhook sender, or another API.
The identity provider authenticates users or services and issues tokens or session information. Examples include Microsoft Entra ID, Auth0, Okta, Duende IdentityServer, Keycloak, cloud identity services, and custom identity systems. The identity provider owns sign-in policy, token issuance, keys, claims, and often multi-factor authentication.
The ASP.NET Core authentication middleware examines the incoming request. It uses configured authentication handlers, called schemes, to validate cookies, bearer tokens, certificates, or other credentials. If successful, it creates the user identity for the request.
The authorization system evaluates whether that identity can access the resource. ASP.NET Core supports role-based and policy-based authorization. Policies can evaluate claims and custom requirements.
The endpoint or controller performs application behavior only after the security boundary allows it. Some endpoints may allow anonymous access, such as health checks or public metadata. Others require an authenticated user or a specific policy.
The data layer receives the result of these decisions indirectly. It should not assume the endpoint already made every correct access decision. Important business rules, such as tenant ownership or row-level access, should be enforced near the application service or data boundary as well.
The configuration system provides non-secret and secret settings. In development, user secrets can hold local values. In production, a managed secret store or platform configuration should provide sensitive values.
Failure boundaries include expired tokens, wrong issuer, wrong audience, missing signing keys, clock skew, missing claims, incorrect policies, confused roles, overbroad permissions, leaked secrets, broken redirect URIs, CORS misconfiguration, and authorization checks that protect the endpoint but not the underlying business operation.

Layer 3 — Core Mechanics

In ASP.NET Core, authentication and authorization are configured in `Program.cs`.

A JWT bearer API has this broad shape:

```
builder.Services
    .AddAuthentication()
    .AddJwtBearer(options =>
    {
        options.Authority = builder.Configuration["Auth:Authority"];
        options.Audience = builder.Configuration["Auth:Audience"];
    });

builder.Services.AddAuthorization();

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

app.MapGet("/orders", () => Results.Ok())
   .RequireAuthorization();

app.Run();
```

`AddAuthentication` registers authentication services. `AddJwtBearer` registers a bearer-token authentication scheme. The authority identifies the issuer trusted by the API. The audience identifies the API the token is meant for.
`AddAuthorization` registers authorization services.
`UseAuthentication` runs authentication middleware. `UseAuthorization` runs authorization middleware. The order matters: authorization depends on the identity created by authentication.
`RequireAuthorization` marks the endpoint as protected.
A policy expresses a named access rule:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CanReadOrders", policy =>
    {
        policy.RequireAuthenticatedUser();
        policy.RequireClaim("scope", "orders.read");
    });
});
```

The endpoint can require that policy:

```
app.MapGet("/orders/{id:int}", (int id) =>
{
    return Results.Ok(new { id });
})
.RequireAuthorization("CanReadOrders");
```

For controllers, the same idea appears with attributes:

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
[Authorize(Policy = "CanReadOrders")]
public sealed class OrdersController : ControllerBase
```

```
{
    [HttpGet("{id:int}")]
    public IActionResult Get(int id)
    {
        return Ok(new { id });
    }
}
```

JWT stands for JSON Web Token. In an API scenario, a bearer token is commonly sent like this:

```
Authorization: Bearer eyJhbGciOi...
```

The API should validate the token's signature, issuer, audience, expiration, and relevant claims. It should not merely decode the token and trust the contents.
Secrets should come from configuration, not constants:

```
var connectionString =
    builder.Configuration.GetConnectionString("Orders");
```

For local development, user secrets can keep values out of the repository:

```
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:Orders" "Server=..."
```

User secrets are for development convenience. They are not a production secret store.

Layer 4 — Developer Workflow

A safe development workflow begins before code is written.
Identify the protected resources:

```
Orders
Invoices
User profile
Administration
Reports
Health checks
```

Identify the callers:

```
Anonymous user
Authenticated customer
Employee
Administrator
Background service
Partner API
```

Define access rules in ordinary language:

```
Customers can read their own orders.
Employees can read orders for assigned regions.
Administrators can change order status.
Background services can write fulfillment events.
Anonymous users can read public health metadata only.
```

Then translate those rules into authentication schemes, policies, claims, roles, and application checks.
Create an API:

```
dotnet new webapi -n Chapter07.Api
cd Chapter07.Api
```

Add JWT bearer support:

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

Initialize user secrets for local values:

```
dotnet user-secrets init
dotnet user-secrets set "Auth:Authority" "https://login.example.com/"
dotnet user-secrets set "Auth:Audience" "chapter07-api"
```

Build and run:

```
dotnet build
dotnet run
```

The exact identity-provider configuration depends on the provider. Microsoft Entra ID, Auth0, Okta, Keycloak, and other providers all require application registration, redirect URI or API audience settings, and token configuration. Those details belong to the provider setup. The ASP.NET Core application should treat the provider as the trusted authority and validate tokens accordingly.
During development, avoid these habits:

- do not commit tokens;
- do not paste production client secrets into local files;
- do not log authorization headers;
- do not turn off token validation to “get unstuck”;
- do not rely on frontend checks as the only authorization enforcement.

Use local test tokens, development identity providers, or integration-test helpers when available. Chapter 8 returns to testing; this chapter’s goal is to establish the secure workflow mindset.

Layer 5 — Production Usage

Production security starts with trust boundaries.
Configuration should separate local, test, staging, and production identity settings. A token issued for a development audience should not work against production. A production API should validate issuer, audience, lifetime, and signature.
Secrets should be stored in a managed secret store or platform secret system. Examples include Azure Key Vault, AWS Secrets Manager, Google Secret Manager, Kubernetes secrets with appropriate encryption and access controls, or a dedicated enterprise vault. Environment variables can be a delivery mechanism, but they are not a complete secret-management strategy by themselves.
Security requires least privilege. The application identity should have only the database, storage, queue, and cloud permissions it needs. Users and service clients should receive only the scopes, roles, or claims required for their work.
Reliability includes key rollover, token expiration, identity-provider availability, clock synchronization, and fallback behavior. If the identity provider has an outage, existing token validation may continue for a period depending on cached metadata and keys, but new sign-ins may fail.
Deployment must protect credentials. CI/CD systems should use secure variable stores, federated identity, managed identity, or short-lived credentials when possible. Long-lived client secrets copied between systems are operational debt.

Observability should record security-relevant events without exposing secrets. Log authentication failures, authorization failures, suspicious request patterns, and administrative actions. Do not log bearer tokens, passwords, client secrets, full connection strings, or sensitive claims.
Scaling affects security because multiple application instances must agree on token validation, data-protection keys for cookie scenarios, clock behavior, and policy configuration. Stateless JWT validation helps APIs scale, but revocation and permission changes require careful design.
Persistence matters because authorization often depends on data ownership. "Can read orders" may not be enough. The application may need to check whether the order belongs to the current tenant, customer, region, or account.
Cost appears through identity-provider licensing, secret store operations, audit-log retention, security scanning, and compliance requirements. Security costs are usually cheaper than incident costs.
Local development optimizes for learning and fast feedback. Production optimizes for verified trust, least privilege, credential hygiene, auditing, and controlled access.

Layer 6 — Tradeoffs and Alternatives

Use ASP.NET Core authentication and authorization for application-level security decisions. Use platform security for infrastructure-level access. Use database security for data-level protection. Strong systems use all three.
Cookies are natural for browser-based web applications where the server and browser maintain an authenticated session. JWT bearer tokens are natural for APIs called by clients, single-page applications, mobile apps, or other services. API keys can be useful for simple service integration, but they do not identify users well and require careful rotation and scope control.
OAuth 2.0 is for delegated authorization. OpenID Connect is for sign-in and identity. Do not describe every token flow as "OAuth login" without knowing which problem the flow solves.
Microsoft Entra ID is common in Microsoft-centered enterprises. Auth0, Okta, Keycloak, Duende IdentityServer, AWS Cognito, Google Identity, and other providers may be better fits depending on organization, hosting platform, customer identity needs, open-source preference, compliance, and operations skills.
ASP.NET Core Identity is useful when an application owns its own user store, especially for traditional web applications. External identity providers are often preferable for enterprise SSO, customer identity, multi-factor authentication, federation, and centralized account governance.
Common overengineering mistakes:

- building a custom identity provider when a managed one would do;
- using roles for every fine-grained business rule;
- trusting decoded JWT payloads without validation;
- putting authorization only in the frontend;
- storing production secrets in `appsettings.json`;
- logging tokens during debugging and forgetting to remove the log;
- using one overpowered service account for every dependency;
- treating authentication as complete security.

State-of-the-art security in modern .NET favors standards-based identity, short-lived tokens, policy-based authorization, managed identities, external secret stores, least privilege, strong telemetry, and threat modeling early in the design.

Layer 7 — Interview Perspective

Interviewers use security questions to test whether you know the difference between identity, access, and secret handling.

Concepts commonly tested:

- authentication versus authorization;
- claims, roles, and policies;
- JWT bearer authentication;
- OAuth 2.0 versus OpenID Connect;
- access tokens versus ID tokens;
- middleware order;
- secret storage;
- least privilege;
- tenant or ownership checks;
- avoiding token and secret leakage.

Representative questions:

- “What is the difference between authentication and authorization?”
- “How does JWT bearer authentication work in an ASP.NET Core API?”
- “Why should an API validate issuer and audience?”
- “What is OpenID Connect’s relationship to OAuth 2.0?”
- “When would you use policies instead of roles?”
- “Where should production secrets be stored?”
- “Why is frontend authorization not enough?”
- “How would you secure an endpoint that returns customer orders?”

A strong answer separates concerns:

> “Authentication establishes the caller’s identity, often through a token or cookie. Authorization decides whether that identity can perform the requested action. For an orders API, I would validate the token, require a policy such as `orders.read`, and still check that the requested order belongs to the caller’s tenant or account.”

Common misconceptions:

- “If a user is authenticated, they can use the endpoint.”
- “JWTs are secure because they are encoded.”
- “OAuth and OpenID Connect are the same thing.”
- “Roles are always enough.”
- “Secrets in environment variables are automatically safe.”
- “HTTPS removes the need for token validation.”
- “The frontend can hide unauthorized buttons, so the API is protected.”

Small design scenario:
You are adding security to the order API from Chapter 6. Customers can read their own orders. Employees can read orders for assigned regions. Administrators can update order status. A background service can add shipment events. A good design would use a trusted identity provider, JWT bearer authentication for the API, policies for scopes or permissions, application-level ownership checks for customer and region access, least-privilege database access, and a secret store for connection strings and identity-provider settings.
The strong answer protects the business operation, not just the URL.

Hands-On Lab

Objective:
Add basic authorization structure and local secret storage to an ASP.NET Core API.

Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- Basic ASP.NET Core experience from Chapter 5.
- No real identity provider is required for the conceptual parts of this lab.

Steps:

1. Create an API:

```
dotnet new webapi -n Chapter07.Api
cd Chapter07.Api
```

2. Add JWT bearer support:

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

3. Initialize user secrets:

```
dotnet user-secrets init
```

4. Store placeholder development settings:

```
dotnet user-secrets set "Auth:Authority" "https://login.example.com/"
dotnet user-secrets set "Auth:Audience" "chapter07-api"
```

5. Add authentication and authorization services in `Program.cs`.
6. Add `UseAuthentication()` before `UseAuthorization()`.
7. Add a policy named `CanReadOrders`.
8. Protect an endpoint with `.RequireAuthorization("CanReadOrders")`.
9. Leave one simple endpoint anonymous, such as `/health/basic`.
10. Build the project:

```
dotnet build
```

Expected results:

- The project builds.
- Authentication and authorization are configured.
- Protected endpoints require authorization.
- Local secret values are stored outside source-controlled files.

Validation commands:

```
dotnet build
dotnet user-secrets list
dotnet run
```

Troubleshooting notes:

- If the app fails on startup, check that configuration keys match the code.
- If all endpoints are protected, confirm the intended anonymous endpoint does not call `RequireAuthorization`.
- If authorization always fails, remember that a real request needs a valid token from the configured authority.
- If `dotnet user-secrets` fails, confirm the command is run from the project directory.

Knowledge Check

1. Why is authentication not enough to protect an API?
2. What does authorization decide?
3. Why should a JWT be validated instead of merely decoded?
4. What is the difference between an access token and an ID token?
5. How does OpenID Connect relate to OAuth 2.0?
6. Why does middleware order matter for authentication and authorization?
7. When are policies better than roles?
8. Why should authorization sometimes check resource ownership, not only claims?
9. What kinds of values should be treated as secrets?
10. Why are user secrets appropriate for development but not production?

Summary

Authentication identifies the caller. Authorization decides what the caller may do. Secrets management protects the sensitive values the application needs in order to run.

ASP.NET Core provides authentication schemes, middleware, authorization policies, endpoint protection, and integration points for modern identity providers. JWT bearer authentication is common for APIs. OAuth 2.0 addresses delegated authorization. OpenID Connect extends OAuth 2.0 for sign-in and identity.

The most important security habit is to protect business operations, not just routes. Validate tokens. Require policies. Check ownership and tenant boundaries. Keep secrets out of code. Use least privilege for users, services, databases, and cloud resources.

The next chapter adds automated testing and CI basics so application behavior, including security-sensitive behavior, can be validated repeatedly before deployment.

Sources

- Overview of ASP.NET Core authentication
- Introduction to authorization in ASP.NET Core
- Configure JWT bearer authentication in ASP.NET Core
- OpenID Connect on the Microsoft identity platform
- Safe storage of app secrets in development in ASP.NET Core
- OWASP API Security Top 10

Further Reading

- Authentication and authorization in Minimal APIs
- Policy-based authorization in ASP.NET Core
- Microsoft identity platform documentation
- Azure Key Vault documentation
- OWASP Cheat Sheet Series

CHAPTER 8

Automated Testing and CI Basics

Chapter Purpose

The previous chapters built the first usable application shape: modern .NET, ASP.NET Core, data access, and security. That is enough to write meaningful software. It is also enough to break meaningful software.

Automated testing and continuous integration exist because professional teams need confidence before change reaches production. Manual testing, careful code review, and developer judgment still matter, but they are not enough by themselves. People forget steps. Local machines drift. Integration problems hide until code meets other code. Security-sensitive behavior can regress quietly. A database change can compile but fail when the application starts.

If your earlier model involved a local Visual Studio build, a QA phase, and a build server that produced occasional releases, the modern model pulls validation earlier. Every proposed change should be able to prove basic health: the solution restores, builds, and passes the relevant test suite in a clean environment.

This chapter introduces automated validation, unit tests, integration tests, test projects, GitHub Actions, pull request checks, and build pipelines. It focuses on validating application changes before deployment. Production release automation comes later in Chapter 16.

Where This Fits

Automated testing and CI sit between developer workflow and deployment.

```
Developer branch
        |
        v
Local build and tests
        |
        v
Pull request
        |
        v
CI workflow
        |
        +-- restore packages
        +-- build solution
        +-- run unit tests
        +-- run selected integration tests
        +-- report status
        |
        v
Merge decision
```

This chapter builds directly on Chapter 3's workflow loop. It uses the application structure from Chapters 4-7 but does not require Linux, Docker, cloud hosting, or production pipeline knowledge yet.

That ordering matters. Before learning deployment automation, the reader should know what a pipeline is supposed to validate. A fast, repeatable build and test process is the foundation. Without it, continuous deployment simply makes broken releases move faster.

Connection to the Reader's Existing Model

You probably already know several older validation patterns:

- compile the solution in Visual Studio;
- run the application locally;
- execute a few manual smoke tests;
- rely on QA test plans;
- use a build server to produce artifacts;
- deploy to a test server;
- fix bugs found after integration.

Modern testing and CI keep the same goal but move feedback closer to the code change.
A unit test is like a small repeatable check around one piece of logic. Instead of opening the application and manually exercising a case, the test runner does it every time.
An integration test is closer to exercising a slice of the application the way the runtime uses it. In ASP.NET Core, integration tests can start a test host and send HTTP requests through the request pipeline.
A CI build is like a clean build machine with discipline. It does not care that the code worked in your Visual Studio session. It restores dependencies, builds, and tests from source in a separate environment.
A pull request check is like an automated reviewer that handles mechanical questions: does it compile, do tests pass, did formatting drift, did a basic security scan fail? It cannot judge whether the design is good, but it can protect the team from many repeatable mistakes.
The analogy breaks down when CI is treated as a final QA department. CI is a feedback system, not a substitute for thoughtful design, exploratory testing, threat modeling, performance testing, or production monitoring.

Layer 1 — Conceptual Model

Automated testing is executable evidence that the software behaves as expected in specific scenarios.
Continuous integration is the practice of automatically validating changes when they are committed or proposed for merge.
Together, they solve these problems:

- catching regressions early;
- proving that the project builds outside one developer's machine;
- reducing integration surprises;
- documenting expected behavior in executable form;
- giving reviewers confidence;
- protecting the main branch;
- making later deployment automation safer.

They do not solve these problems automatically:

- they do not prove the absence of bugs;
- they do not replace human review;
- they do not guarantee useful coverage;
- they do not validate production performance by default;

- they do not make flaky tests trustworthy;
- they do not remove the need for observability after deployment.

The conceptual model is:

```
Unit tests answer: does this small behavior work?
Integration tests answer: do these components work together?
CI answers: does this change work from a clean shared process?
Pull request checks answer: is this change ready for human merge judgment?
```

The goal is not to maximize test count. The goal is to create a fast, trustworthy signal about the risks that matter.

Layer 2 — System Relationships

The production project contains application code. It might be an ASP.NET Core API, class library, worker service, or console application.
The test project contains test code. It references the production project and uses a test framework such as MSTest, xUnit, or NUnit.
The test framework provides attributes, assertions, fixtures, and conventions. MSTest, xUnit, and NUnit are common choices in .NET. The best choice is often the one your team already understands.
The test runner discovers and executes tests. `dotnet test` builds the project and runs tests using the configured runner. In .NET 10, test runner selection can be configured, with VSTest as the traditional default and Microsoft Testing Platform available for newer scenarios.
The system under test, often shortened to SUT, is the application or component being tested. In an ASP.NET Core integration test, the SUT is usually the web application started by the test host.
Test doubles replace dependencies when isolation is useful. Fakes, stubs, mocks, and in-memory implementations can make unit tests fast and focused. They can also hide integration problems when overused.
The CI system runs validation away from the developer's workstation. GitHub Actions, Azure Pipelines, GitLab CI, Jenkins, TeamCity, CircleCI, and others can all perform this role.
The pull request receives CI status. Branch protection can require selected checks to pass before merge.
Failure boundaries include missing SDKs, package restore failures, flaky tests, tests that depend on local machine state, slow integration tests, secrets missing in CI, inconsistent databases, incorrect test data, weak assertions, and test suites that are ignored because they are noisy.

Layer 3 — Core Mechanics

A unit test starts with a small behavior.

```
public sealed class OrderTotalCalculator
{
    public decimal Calculate(decimal subtotal, decimal tax)
    {
        if (subtotal < 0)
        {
            throw new ArgumentOutOfRangeException(nameof(subtotal));
        }

        return subtotal + tax;
    }
}
```

An MSTest unit test might look like this:

```
using Microsoft.VisualStudio.TestTools.UnitTesting;

[TestClass]
public sealed class OrderTotalCalculatorTests
{
    [TestMethod]
    public void Calculate_WithSubtotalAndTax_ReturnsTotal()
    {
        var calculator = new OrderTotalCalculator();

        var result = calculator.Calculate(100m, 8.25m);

        Assert.AreEqual(108.25m, result);
    }
}
```

The familiar pattern is Arrange, Act, Assert:

```
Arrange: create the situation
Act: run the behavior
Assert: verify the result
```

An ASP.NET Core integration test checks a broader slice. It can start the app in a test host and make HTTP requests through the pipeline:

```
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.VisualStudio.TestTools.UnitTesting;

[TestClass]
public sealed class HealthEndpointTests
{
    [TestMethod]
    public async Task HealthEndpoint_ReturnsOk()
    {
        await using var factory =
            new WebApplicationFactory<Program>();

        using var client = factory.CreateClient();

        var response = await client.GetAsync("/health/basic");

        response.EnsureSuccessStatusCode();
    }
}
```

This type of test can catch routing, middleware, dependency injection, and configuration problems that a small unit test would miss.

A basic GitHub Actions workflow for .NET validation has the same shape as the local commands:

```
name: dotnet-ci

on:
  pull_request:
  push:
    branches: [ "main" ]
```

```
jobs:
  build-test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v6

      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: "10.0.x"

      - run: dotnet restore

      - run: dotnet build --no-restore --configuration Release

      - run: dotnet test --no-build --configuration Release
```

The YAML is not magic. It checks out code, installs or selects .NET, restores packages, builds, and runs tests.

Layer 4 — Developer Workflow

Start by creating a solution with production and test projects:

```
dotnet new sln -n Chapter08
dotnet new classlib -n Chapter08.Domain
dotnet new mstest -n Chapter08.Domain.Tests
dotnet sln add Chapter08.Domain/Chapter08.Domain.csproj
dotnet sln add Chapter08.Domain.Tests/Chapter08.Domain.Tests.csproj
dotnet add Chapter08.Domain.Tests/Chapter08.Domain.Tests.csproj reference Chapter08.Domain/
Chapter08.Domain.csproj
```

Run tests locally:

```
dotnet test
```

Run with Release configuration:

```
dotnet test --configuration Release
```

Filter tests when diagnosing:

```
dotnet test --filter "FullyQualifiedName~Order"
```

Create an ASP.NET Core integration test project when testing the request pipeline:

```
dotnet new mstest -n Chapter08.Api.Tests
dotnet add Chapter08.Api.Tests package Microsoft.AspNetCore.Mvc.Testing
dotnet add Chapter08.Api.Tests reference Chapter08.Api/Chapter08.Api.csproj
dotnet sln add Chapter08.Api.Tests/Chapter08.Api.Tests.csproj
```

Keep unit and integration tests separated when practical. This makes it easier to run fast tests frequently and broader tests intentionally.
Create a workflow file:

```
.github/workflows/dotnet-ci.yml
```

Use it to run restore, build, and test on pull requests and pushes to `main`.
The daily workflow becomes:

```
Change code
  |
  v
Run focused tests locally
  |
  v
Run full local validation before opening PR
  |
  v
Open PR
  |
  v
CI validates in clean environment
  |
  v
Reviewer evaluates behavior and design
```

When CI fails, assume the signal matters until you prove otherwise. A flaky test is not harmless. It teaches the team to distrust the pipeline.

Layer 5 — Production Usage

This chapter is not about production deployment, but test and CI choices shape production risk.

Configuration should distinguish local test settings from production settings. Tests should not accidentally connect to production databases, production queues, production identity providers, or paid AI services.

Secrets should be avoided in tests when possible. When CI needs secrets, use the CI platform's secure secret store and restrict access. Never commit secrets to test configuration.

Security-sensitive behavior deserves tests. Authorization policies, ownership checks, input validation, and secret-loading behavior should not rely only on manual testing. Chapter 7's security boundaries are good candidates for focused integration tests.

Reliability improves when CI protects the main branch. A main branch that regularly fails builds becomes background noise. A healthy main branch gives the team a trustworthy integration point.

Deployment depends on validated artifacts. Later CD pipelines should consume artifacts produced after restore, build, and test. If the validation path is unclear, deployment automation inherits that uncertainty.

Observability starts in CI too. Failed tests, test duration, flaky test patterns, code coverage trends, and build times are signals about the health of the engineering system.

Scaling the test suite requires layers. Thousands of slow integration tests on every pull request may block development. A better shape is many fast unit tests, focused integration tests for important boundaries, and heavier tests scheduled or gated where appropriate.

Persistence in tests requires caution. Tests that write to a real database need clean setup and teardown, isolated data, or disposable databases. Shared test databases often create order-dependent failures.

Cost appears through CI minutes, hosted runners, cloud test environments, database instances, browser tests, and AI-assisted test generation. Fast feedback is valuable; wasteful feedback is still waste.

Local development optimizes for speed. CI optimizes for clean reproducibility. Production optimizes for safety. The three should align, but they are not the same environment.

Layer 6 — Tradeoffs and Alternatives

Use unit tests for deterministic business rules, validation logic, mapping logic, calculations, authorization decisions, and small service behavior.
Use integration tests for ASP.NET Core routing, middleware, dependency injection, database access, configuration, authentication behavior, and important cross-component flows.
Use end-to-end tests sparingly for complete user or system flows. They are valuable, but slower and more fragile. Browser automation tools such as Playwright are useful when the UI matters.
MSTest, xUnit, and NUnit are all credible .NET test frameworks. MSTest has first-party Microsoft positioning and integrates naturally with Visual Studio. xUnit is common in open-source and ASP.NET Core samples. NUnit is mature and widely used. Pick one deliberately and avoid mixing frameworks casually.
GitHub Actions is a natural CI choice for repositories hosted on GitHub. Azure Pipelines fits many Azure DevOps organizations. GitLab CI, Jenkins, TeamCity, CircleCI, and Buildkite are valid alternatives depending on existing platform, compliance, hosting, and team skill.
Common overengineering mistakes:

- testing trivial property getters while missing business rules;
- mocking every dependency until the test no longer represents reality;
- writing only integration tests and accepting slow feedback;
- depending on test execution order;
- allowing flaky tests to remain unresolved;
- treating code coverage percentage as proof of quality;
- running production-like tests against production resources;
- adding CI gates no one understands.

The state-of-the-art direction is faster, more reliable feedback: clear test layers, parallel execution where safe, disposable infrastructure for integration tests, test impact analysis in large systems, and CI workflows that produce useful diagnostics instead of only red or green lights.

Layer 7 — Interview Perspective

Interviewers use testing and CI questions to learn whether you can work safely on a team.
Concepts commonly tested:

- unit tests versus integration tests;
- Arrange, Act, Assert;
- test project structure;
- `dotnet test`;
- test doubles;
- CI workflow basics;
- pull request checks;
- branch protection;
- flaky tests;
- testing security and data-access behavior.

Representative questions:

- “What is the difference between a unit test and an integration test?”
- “What should run in CI for a .NET API?”
- “How would you structure test projects in a solution?”
- “What do you do when a test passes locally but fails in CI?”

- “Why are flaky tests dangerous?”
- “How would you test an authorization policy?”
- “What belongs in a pull request check versus a deployment pipeline?”

A strong answer connects tests to risk:

> “I use unit tests for fast feedback on isolated business behavior and focused integration tests for boundaries such as HTTP routing, dependency injection, database access, and authorization. CI should restore, build, and run the relevant tests in a clean environment before merge.”

Common misconceptions:

- “Unit tests prove the whole application works.”
- “Integration tests are always better because they test more.”
- “CI is only useful after deployment automation exists.”
- “Flaky tests are acceptable if they usually pass.”
- “Code coverage means the behavior is correct.”
- “Tests should connect to whatever database is easiest.”
- “A pull request check replaces code review.”

Small design scenario:
You inherit an ASP.NET Core order API with EF Core and JWT authorization. The team manually tests endpoints before release and often finds broken authorization or migration problems late.
A good first validation plan would add unit tests for order business rules, integration tests for protected endpoints, a migration check for pending model changes, and a GitHub Actions workflow that restores, builds, and runs tests on pull requests. It would not begin by building a complex production release pipeline.

Hands-On Lab

Objective:
Create a test project, write a unit test, and add a basic GitHub Actions CI workflow for a .NET solution.
Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- Git installed.
- A GitHub repository if you want to run the workflow remotely.

Steps:

1. Create a solution and projects:

```
dotnet new sln -n Chapter08
dotnet new classlib -n Chapter08.Domain
dotnet new mstest -n Chapter08.Domain.Tests
dotnet sln add Chapter08.Domain/Chapter08.Domain.csproj
dotnet sln add Chapter08.Domain.Tests/Chapter08.Domain.Tests.csproj
dotnet add Chapter08.Domain.Tests/Chapter08.Domain.Tests.csproj reference
Chapter08.Domain/Chapter08.Domain.csproj
```

2. Add a small class to the domain project:

```
public sealed class OrderTotalCalculator
{
    public decimal Calculate(decimal subtotal, decimal tax)
```

```
    {
        if (subtotal < 0)
        {
            throw new ArgumentOutOfRangeException(nameof(subtotal));
        }

        return subtotal + tax;
    }
}
```

3. Add a test for the calculator.
4. Run tests:

```
dotnet test
```

5. Create `.github/workflows/dotnet-ci.yml`.
6. Add this workflow:

```
name: dotnet-ci

on:
  pull_request:
  push:
    branches: [ "main" ]

jobs:
  build-test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v6

      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: "10.0.x"

      - run: dotnet restore
      - run: dotnet build --no-restore --configuration Release
      - run: dotnet test --no-build --configuration Release
```

7. Commit the workflow and open a pull request if using GitHub.

Expected results:

- The solution builds.
- The test project references the production project.
- `dotnet test` runs the test locally.
- GitHub Actions can restore, build, and test the solution on pull requests.

Validation commands:

```
dotnet restore
dotnet build
dotnet test
git status
```

Troubleshooting notes:

- If the test project cannot see the production type, check the project reference.
- If CI fails but local tests pass, compare SDK versions and operating systems.
- If the workflow cannot find a solution, confirm it runs from the repository root.
- If `actions/setup-dotnet` cannot find the version, check the supported SDK version syntax.

Knowledge Check

1. Why does automated validation belong before deployment automation?
2. What is the difference between a unit test and an integration test?
3. Why should unit tests usually avoid real databases and network calls?
4. When is an integration test worth its extra cost?
5. What does `dotnet test` do?
6. Why is a clean CI environment more trustworthy than one developer's machine?
7. What should a basic .NET pull request workflow validate?
8. Why are flaky tests harmful to team behavior?
9. What security-sensitive behavior should be tested in an API?
10. Why is code coverage useful but insufficient?

Summary

Automated testing turns expected behavior into executable evidence. Continuous integration runs that evidence in a clean shared process before changes are merged.

Unit tests are fast checks around small behavior. Integration tests verify that important components work together, such as ASP.NET Core routing, middleware, dependency injection, configuration, data access, and authorization. CI ties those checks to pull requests and the main branch.

The goal is not a giant test suite for its own sake. The goal is trustworthy feedback. A healthy validation system is fast enough to use, broad enough to catch meaningful regressions, and reliable enough that the team believes it.

The next chapter shifts operating systems. With a validated .NET application workflow in place, you can now learn the Linux concepts modern .NET developers need before containers and cloud hosting.

Sources

- dotnet test command
- Write tests with MSTest
- Integration tests in ASP.NET Core
- Building and testing .NET with GitHub Actions
- GitHub Actions documentation
- Order unit tests

Further Reading

- Testing in .NET
- Unit testing C# with MSTest and .NET
- ASP.NET Core testing documentation
- Workflow syntax for GitHub Actions

CHAPTER 9

Linux for .NET Developers

Chapter Purpose

Modern .NET is cross-platform, which means a .NET developer can no longer assume that production is always Windows Server and IIS.

That does not mean you must become a full-time Linux administrator. This chapter has a narrower goal: give an experienced Windows and .NET developer enough Linux fluency to understand where modern .NET applications run, how to navigate a Linux environment, how processes and permissions work, how basic networking is inspected, and why Linux matters before Docker and cloud hosting.

Linux exists because the software industry needed an open, Unix-like operating system that could run across many hardware and server environments. It became the dominant operating-system foundation for cloud infrastructure, containers, DevOps tooling, and open-source server software. Microsoft did not make .NET cross-platform by accident. .NET had to meet the operating system where much of modern server computing already lived.

For the reader of this book, Linux is not a replacement for everything you know about Windows. It is a second operating-system mental model. You already understand processes, services, files, permissions, networking, logs, and deployment. This chapter translates those ideas.

Where This Fits

Linux is the operating-system layer underneath many modern .NET runtime environments.

```
ASP.NET Core application
        |
        v
.NET runtime
        |
        v
Linux process
        |
        +-- filesystem
        +-- users and permissions
        +-- environment variables
        +-- sockets and ports
        +-- logs and service manager
        |
        v
VM, container host, cloud app platform, or Kubernetes node
```

Chapter 10 introduces Docker. Docker will make much more sense if Linux processes, filesystems, users, ports, and environment variables are already familiar. Chapter 12 returns to hosting. Chapter 18 introduces Kubernetes, where Linux concepts appear again underneath pods and containers.

This chapter is about orientation, not mastery. You need enough Linux to read commands, diagnose common hosting problems, and avoid Windows assumptions.

Connection to the Reader's Existing Model

Windows and Linux solve similar operating-system problems with different conventions.
A Windows process maps to a Linux process. Both have process IDs, memory, threads, environment variables, open files, and network connections.
A Windows service maps roughly to a `systemd` service on many Linux distributions. Both define long-running background processes that start, stop, restart, and write logs. The analogy is useful, but Linux distributions can vary, and containers often run a process directly without `systemd`.
An IIS application pool identity maps to a Linux user account or service user. Both determine what the running process can access. The difference is that Linux file permissions and ownership are visible almost everywhere and are central to daily troubleshooting.
`C:\inetpub\wwwroot` maps conceptually to an application directory such as `/var/www/myapp`, `/opt/myapp`, or a container working directory. The Linux filesystem does not use drive letters. Everything hangs from `/`, the root of the filesystem.
Windows Event Log maps partly to `journalctl`, partly to application console logs, and partly to files under `/var/log`. In containers and cloud platforms, writing structured logs to standard output is often preferred.
Windows environment variables map directly to Linux environment variables, but syntax differs. PowerShell uses `$env:ASPNETCORE_ENVIRONMENT`; shells such as Bash use `$ASPNETCORE_ENVIRONMENT`.
The analogy breaks down when you assume the same defaults. Linux paths are case-sensitive on most filesystems. `/app/config.json` and `/app/Config.json` are different names. File execution depends on permission bits. Services often run as non-root users. Package installation, certificates, process management, and shell behavior are different enough to deserve attention.

Layer 1 — Conceptual Model

Linux is an operating-system family built around the Linux kernel plus a userspace of tools, libraries, shells, package managers, and distributions.
For .NET developers, Linux solves these practical problems:

- it provides a common server runtime environment;
- it underpins most container hosts;
- it is widely used by cloud platforms;
- it supports command-line automation well;
- it gives lightweight server images for APIs and services;
- it provides a common environment for CI runners and deployment targets.

It does not solve these problems by itself:

- it does not make an application cloud-native;
- it does not remove the need for deployment discipline;
- it does not make security automatic;
- it does not replace SQL Server, IIS, or Windows where those are still the right fit;
- it does not make every Windows-specific .NET application portable.

The conceptual center is this:

```
Linux hosts processes.
Processes read files, environment variables, ports, and permissions.
.NET applications are processes.
```

Once you understand that, Linux becomes less mysterious. An ASP.NET Core app on Linux is still a process listening on a port, reading configuration, calling dependencies, writing logs, and exiting with a status code.

Layer 2 — System Relationships

The Linux kernel owns low-level resources: CPU scheduling, memory, processes, filesystems, devices, networking, and permissions.

The distribution packages the kernel with userspace tools and conventions. Ubuntu, Debian, Red Hat Enterprise Linux, Fedora, Alpine, SUSE, and others are Linux distributions. They differ in package managers, default tools, support policies, filesystem layout details, and operational conventions.

The shell receives commands from the user or script. Bash is common, though many shells exist. The shell expands variables, runs programs, redirects input/output, and connects commands.

The package manager installs and updates software. Debian and Ubuntu commonly use `apt`. Red Hat-family systems commonly use `dnf` or `yum`. Alpine uses `apk`. .NET installation instructions vary by distribution.

The filesystem stores application files, configuration, logs, certificates, temporary data, and mounted volumes. Linux uses `/` as the root and paths such as `/etc`, `/var`, `/usr`, `/home`, `/tmp`, and `/opt`.

Users and groups define ownership and access. A process runs as a user. That user's permissions determine whether the process can read a file, write a directory, bind to certain ports, or access mounted resources.

Networking exposes sockets and ports. An ASP.NET Core process may listen on `http://0.0.0.0:8080` inside a Linux host or container. A firewall, reverse proxy, container port mapping, or cloud load balancer may determine whether clients can reach it.

The .NET runtime executes the application. Microsoft provides .NET downloads and installation guidance for Windows, macOS, and many Linux distributions, and .NET 10 downloads include Linux SDK/runtime builds for multiple architectures.

Failure boundaries include missing packages, wrong CPU architecture, file permission errors, case-sensitive path bugs, missing environment variables, ports already in use, certificate trust problems, line-ending issues, native library dependencies, and differences between a full VM and a minimal container image.

Layer 3 — Core Mechanics

The filesystem starts at `/`.

```
/              root of the filesystem
/home          user home directories
/etc           system configuration
/var           variable data such as logs and application state
/tmp           temporary files
/usr           installed user-space programs and libraries
/opt           optional or vendor application software
```

Paths use forward slashes:

```
/home/steve/appsettings.json
/var/log/myapp/app.log
/opt/orders-api/Orders.Api.dll
```

Permissions are commonly shown with `ls -l`:

```
-rw-r--r--  1 app app  1024 Jul 22 appsettings.json
drwxr-xr-x  2 app app  4096 Jul 22 logs
```

The first character indicates type: `-` for file, `d` for directory. The next groups represent read, write, and execute permissions for owner, group, and others.
Processes can be inspected:

```
ps aux
ps -ef
```

Ports can be inspected:

```
ss -tulpn
```

Environment variables can be viewed:

```
printenv
echo "$ASPNETCORE_ENVIRONMENT"
```

A .NET app can run directly:

```
dotnet Orders.Api.dll
```

Or a published self-contained app can run as an executable:

```
./Orders.Api
```

The execute bit matters. If Linux says “permission denied” when running a file, it may not be executable:

```
chmod +x Orders.Api
```

Logs may come from the process output:

```
dotnet Orders.Api.dll
```

Or from `systemd` when the app is configured as a service:

```
journalctl -u orders-api
```

These commands are not the whole Linux world. They are the beginning of the Linux vocabulary a .NET developer needs.

Layer 4 — Developer Workflow

Start by identifying the environment:

```
uname -a
cat /etc/os-release
whoami
pwd
```

Navigate the filesystem:

```
ls
ls -la
cd /tmp
mkdir chapter09
cd chapter09
```

Create and inspect a file:

```
echo "hello from linux" > message.txt
cat message.txt
ls -l message.txt
```

Inspect environment variables:

```
printenv | sort
export ASPNETCORE_ENVIRONMENT=Development
echo "$ASPNETCORE_ENVIRONMENT"
```

Check .NET:

```
dotnet --info
dotnet --list-runtimes
```

Create and run a .NET app:

```
dotnet new console -n LinuxDotnetDemo
cd LinuxDotnetDemo
dotnet run
```

Publish it:

```
dotnet publish -c Release
```

Find files:

```
find . -name "*.dll"
```

Inspect running processes:

```
ps -ef | grep dotnet
```

The workflow habit is to ask four questions:

```
Where am I in the filesystem?
Which user am I?
Which process is running?
Which port or file is it trying to use?
```

Those four questions solve a surprising number of first-level Linux hosting problems.

Layer 5 — Production Usage

Production Linux hosting starts with ownership.

Do not run application processes as `root` unless there is a specific and justified reason. Create a service user or use the platform's non-root runtime model. File permissions should allow the application to read what it needs and write only where it is expected to write.

Configuration should come from files, environment variables, secret stores, or platform configuration, depending on the host. Avoid editing production application binaries or source files on the server.

Secrets should not be stored in shell history, world-readable files, or checked-in scripts. Linux permissions help, but a readable secret file is still a secret exposure if too many users or processes can read it.

Security includes patching, user permissions, firewall rules, SSH access, certificate trust, dependency updates, and process isolation. Linux is not secure merely because it is Linux.

Reliability depends on service management, restart behavior, logs, health checks, disk space, memory pressure, CPU pressure, and dependency availability. On VM-style Linux hosting, `systemd` commonly owns service restart behavior. In containers, the container platform usually owns restart behavior.

Deployment should be repeatable. Copying files manually into `/opt` may be acceptable for learning, but production should use packages, artifacts, containers, or deployment automation.

Observability depends on where logs go. A VM service may write to journald and application logs. A containerized app should usually write logs to standard output so the platform can collect them.
Scaling is usually handled outside the individual Linux process: multiple VMs, containers, platform instances, or Kubernetes pods. The application still needs to be stateless enough to scale safely.
Persistence should be explicit. Local Linux disk may be temporary in cloud or container environments. Durable state belongs in databases, managed storage, or mounted volumes designed for persistence.
Cost is affected by VM size, disk, network traffic, operational labor, support subscriptions, and cloud platform choices. Linux can reduce licensing cost in some scenarios, but it does not eliminate operations cost.
Local development optimizes for learning and convenience. Production optimizes for least privilege, patching, repeatability, monitoring, and recoverability.

Layer 6 — Tradeoffs and Alternatives

Use Linux hosting when the application benefits from container alignment, cloud-native platform support, lower server footprint, open-source tooling, or team familiarity with Linux operations.
Use Windows hosting when the application depends on Windows-specific APIs, classic .NET Framework, COM components, Windows Authentication patterns, desktop automation, vendor software, or an operations team and compliance model built around Windows Server.
Use managed platforms when you do not want to administer Linux VMs directly. Azure App Service, Azure Container Apps, AWS Elastic Beanstalk, AWS ECS, Google Cloud Run, and similar services can run .NET workloads while hiding some operating-system management.
Use containers when repeatable runtime packaging matters. Containers usually run on Linux hosts, though Windows containers exist for Windows-specific workloads.
Linux distributions are alternatives too. Ubuntu is common in cloud and developer contexts. Debian is common for stability and base images. Red Hat Enterprise Linux is common in enterprises with support requirements. Alpine is common for small container images, though its musl C library can introduce compatibility considerations. The right choice depends on support, image size, package availability, security policy, and team experience.
Common overengineering mistakes:

- trying to become a Linux administrator before learning the basics needed for .NET hosting;
- running everything as `root` to avoid permission errors;
- assuming paths are case-insensitive;
- editing production files manually instead of redeploying;
- storing secrets in shell scripts;
- ignoring logs because they are not in Windows Event Viewer;
- choosing a minimal container image before understanding missing dependencies.

The state-of-the-art direction is not “Linux instead of Windows everywhere.” It is platform fit: Linux for cloud-native and container-friendly workloads, Windows where Windows-specific value remains, and managed platforms where the team should not own the OS layer directly.

Layer 7 — Interview Perspective

Interviewers usually do not expect a .NET developer to be a senior Linux administrator. They do expect comfort with the basics.
Concepts commonly tested:

- why Linux matters to modern .NET;
- filesystem layout and path differences;

- case-sensitive paths;
- users, groups, and permissions;
- processes and services;
- environment variables;
- ports and networking;
- logs and standard output;
- Linux in containers and cloud hosting;
- Windows versus Linux hosting tradeoffs.

Representative questions:

- “Why would an ASP.NET Core app run on Linux?”
- “What problems can case-sensitive paths cause?”
- “How would you check whether a .NET process is running?”
- “How would you inspect environment variables?”
- “What does `permission denied` usually suggest?”
- “Where would you expect logs to go on Linux?”
- “When would Windows Server still be the better host?”

A strong answer translates from concepts:

> “An ASP.NET Core app on Linux is still a process. I would check which user it runs as, which directory it uses, whether required environment variables are set, whether it can read its files, and whether it is listening on the expected port.”

Common misconceptions:

- “Linux is required for modern .NET.”
- “Linux knowledge means memorizing every command.”
- “Containers mean I do not need to understand Linux.”
- “If a file path works on Windows, it will work the same on Linux.”
- “Running as root is fine if the server is private.”
- “Linux logs all appear in one obvious place.”

Small design scenario:
You deploy an ASP.NET Core API to a Linux VM. It starts locally on Windows but fails in production. The logs say it cannot find `AppSettings.json`.
A good investigation would check the deployed filename, the code’s expected path, Linux case sensitivity, working directory, file permissions, and whether configuration should come from environment variables instead of a copied file.
The strong answer avoids blaming Linux and inspects the operating-system boundary.

Hands-On Lab

Objective:
Practice the Linux commands and concepts most relevant to running a .NET application.
Prerequisites:

- A Linux shell. This may be WSL, a Linux VM, a cloud shell, a container shell, or a native Linux machine.
- .NET SDK installed if you want to run the .NET steps.

Steps:

1. Identify the system:

```
uname -a
cat /etc/os-release
```

2. Identify your user and location:

```
whoami
pwd
```

3. Create a working directory:

```
mkdir -p ~/chapter09
cd ~/chapter09
```

4. Create and inspect files:

```
echo "hello" > message.txt
ls -la
cat message.txt
```

5. Inspect permissions:

```
ls -l message.txt
```

6. Work with an environment variable:

```
export ASPNETCORE_ENVIRONMENT=Development
echo "$ASPNETCORE_ENVIRONMENT"
```

7. Check .NET:

```
dotnet --info
```

8. Create and run a console app:

```
dotnet new console -n LinuxDotnetDemo
cd LinuxDotnetDemo
dotnet run
```

9. Inspect processes:

```
ps -ef | grep dotnet
```

Expected results:

- You can identify the Linux distribution.
- You can navigate directories and inspect files.
- You can read permission output.
- You can set and read an environment variable.
- You can run a .NET application from a Linux shell.

Validation commands:

```
uname -a
cat /etc/os-release
whoami
pwd
ls -la
dotnet --info
dotnet run
```

Troubleshooting notes:

- If `dotnet` is not found, install the .NET SDK for your distribution or use a shell where it is already installed.
- If a command says `permission denied`, check file permissions and the current user.
- If a path is not found, check spelling and capitalization.
- If `grep dotnet` shows only the `grep` command, no matching .NET process may be running.

Knowledge Check

1. Why does Linux matter to modern .NET development?
2. What is the practical difference between `C:\apps\api` and `/opt/api`?
3. Why can filename capitalization break a .NET app on Linux?
4. What does a Linux user account control for a running process?
5. How are environment variables useful for ASP.NET Core configuration?
6. Why should production services usually avoid running as `root`?
7. How do Linux logs differ between VM-style hosting and container hosting?
8. What is the relationship between Linux and Docker?
9. When is Windows Server still the right hosting choice?
10. What four questions should you ask first when diagnosing a Linux-hosted .NET app?

Summary

Linux is not a detour from modern .NET. It is one of the operating-system foundations underneath modern .NET hosting, containers, CI runners, cloud platforms, and Kubernetes.

For a Windows developer, the most useful first step is translation. Linux has processes, services, files, permissions, environment variables, ports, and logs just as Windows does, but the conventions are different. Paths start at `/`, filenames are usually case-sensitive, permissions are visible and central, and many hosting workflows assume shell-based automation.

You do not need to become a Linux administrator before writing .NET applications. You do need enough Linux fluency to understand where the process runs, which user owns it, which files it can read, which port it listens on, and where logs and configuration come from.

The next chapter builds on this by introducing Docker, where Linux process and filesystem concepts become the foundation for images and containers.

Sources

- Install .NET on Windows, Linux, and macOS
- Download .NET 10.0
- The Linux Foundation: It's not just the Linux operating system
- About the Linux Foundation
- Stack Overflow Developer Survey 2025

Further Reading

- Install .NET on Ubuntu

- Install .NET on Debian
- Install .NET on Red Hat Enterprise Linux
- Linux Foundation Training
- Linux.com